1차 과제

실습 환경 구축 및 PyTorch Tensor 기초

실습 보고서

Part 1~5 순서에 따른 코드, 실행 결과, 원리 및 오류 분석

이름	이영언
학번	2024111771
실습 환경	Windows, Python 3.11.9, PyTorch CPU

실습 요약

Python 3.11 가상환경과 uv, Jupyter Lab을 구성하고 PyTorch Tensor의 생성, 속성, 인덱싱, 구조 변경, 기본 연산을 실습했다. CUDA를 사용할 수 없는 환경이므로 과제의 대체 지침에 따라 CPU에서 계산했으며, 실행 화면과 각 결과의 의미를 함께 기록했다.

Part 1. 실습 환경 구축

1.1 Python 설치 확인, 1.2 uv와 PATH, 1.3 Python 3.11 가상환경

1.1 Python 3.11 설치 확인

python.org에서 Windows용 64비트 Python 3.11을 설치한 뒤 PowerShell에서 py - 3.11 - - version과 py - 3.11 - m pip - - version을 실행했다. 첫 명령은 Python Launcher가 3.11 인터프리터를 선택하는지 확인하고, 두 번째는 그 인터프리터에 연결된 pip의 버전과 설치 경로를 확인한다. 캡처에서 Python 3.11.9 및 Python 3.11 경로의 pip가 출력되어 설치가 확인된다.

1.2 uv 설치와 PATH 설정

```
py -3.11 -m pip install uv
uv --version
```

uv는 프로젝트별 가상환경과 패키지 설치를 관리하는 도구다. 처음 uv 명령이 인식되지 않은 것은 uv.exe가 설치된 Python 3.11의 Scripts 폴더가 PATH에 없었기 때문이다. 캡처에서 확인한 경로는 Python311\Scripts이며, 이 폴더를 사용자 PATH에 추가한 뒤 새 PowerShell에서 uv - - version을 실행했다. 출력된 uv 0.13.7로 명령 검색과 설치를 확인했다.

1.3 프로젝트 가상환경 생성 및 활성화

```
cd C:\Users\leeye\Documents\pytorch_tensor_lab
uv venv .venv --python=3.11
.\.venv\Scripts\Activate.ps1
python --version
deactivate
python --version
```

프로젝트 폴더에서 uv venv로 .venv 가상환경을 만들었다. 활성화 전후로 Python 버전을 비교하면 활성화 상태에서는 3.11.9가 사용되고, deactivate 후에는 시스템에 등록된 기본 Python 2.7.17로 돌아간다. 이 비교는 가상환경이 시스템 Python을 덮어쓰지 않고 프로젝트 실행에 사용할 인터프리터만 분리한다는 점을 보여준다.

> 실습 코드와 의존성 설치는 반드시 활성화된 .venv 안에서 실행한다. 프롬프트 앞의 (.venv) 표시와 프로젝트 경로는 본인 환경을 확인하는 증거다.

Part 1. 실습 환경 구축

1.4 requirements.txt 작성, 1.5 PyTorch와 나머지 패키지 설치, Jupyter Lab 확인

1.4 requirements.txt 전체 내용

LLMs-from-scratch의 의존성 목록을 프로젝트 폴더의 requirements.txt로 내려받아 사용했다. 운영체제 조건부 표기는 Intel macOS에서 적용할 Torch/TensorFlow 버전과 그 외 환경에서 적용할 버전을 구분한다. Windows x64 실습 환경에서는 "darwin이 아님" 인 조건의 항목이 적용된다.

```
torch>=2.2.2,<2.6; sys_platform == 'darwin' and platform_machine == 'x86_64'  # Intel macOS
torch >= 2.2.2; sys_platform != 'darwin' or platform_machine != 'x86_64'  # all chapters
jupyterlab >= 4.0  # all
tiktoken >= 0.5.1  # ch02; ch04; ch05
matplotlib >= 3.7.1  # ch04; ch06; ch07
tensorflow>=2.16.2; sys_platform == 'darwin' and platform_machine == 'x86_64'  # Intel macOS
tensorflow >= 2.18.0; sys_platform != 'darwin' or platform_machine != 'x86_64'  # ch05; ch06; ch07
tqdm >= 4.66.1  # ch05; ch07
numpy >= 1.26  # dependency of several other libraries like torch and pandas
pandas >= 2.2.1  # ch06
psutil >= 5.9.5  # ch07; already installed automatically as dependency of torch
```

1.5 CPU PyTorch 설치와 의존성 설치

실습 장치에서 torch.cuda.is_available()이 False였으므로 CUDA 전용 설치가 아니라 CPU wheel 인덱스를 사용했다. GPU가 있는 Windows 환경이라면 과제 안내의 cu121 인덱스를 사용해야 하지만, 이 환경에서는 CPU 설치가 지침에 맞는 선택이다.

```
uv pip install --index-url https://download.pytorch.org/whl/cpu torch torchvision torchaudio
uv pip install -r requirements.txt
uv run jupyter lab
```

첫 명령은 PyTorch와 TorchVision, TorchAudio의 CPU 빌드를 설치하고, 두 번째 명령은 requirements.txt의 나머지 의존성을 .venv에 설치한다. 마지막 명령은 해당 프로젝트 환경의 Jupyter Lab을 실행한다. 브라우저에서 localhost:8888이 열리고 Python 3 커널 및 노트북을 생성한 화면으로 실행을 확인했다.

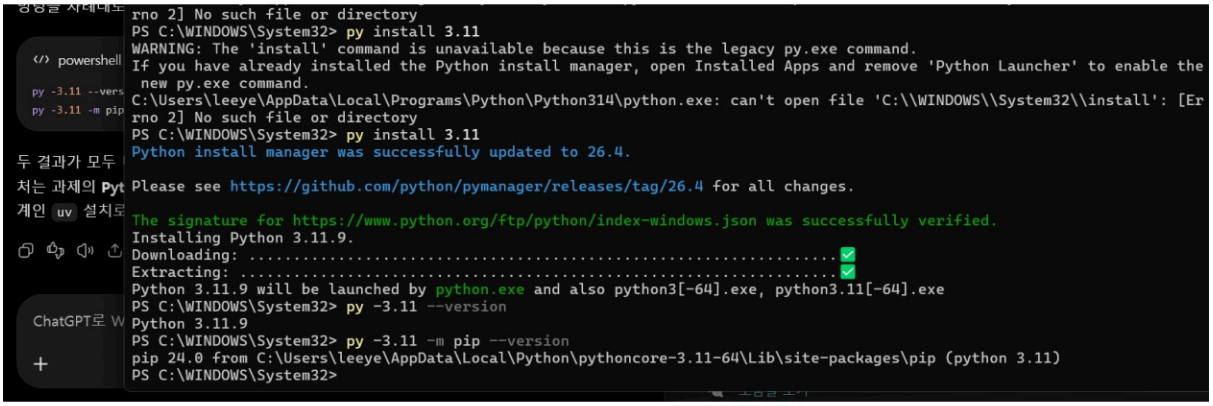
> 이 보고서에는 requirements.txt 전문을 텍스트로 포함했고, 해당 파일이 열려 있는 본인 JupyterLab 화면도 증거로 보존했다.

Part 1. 실습 환경 구축

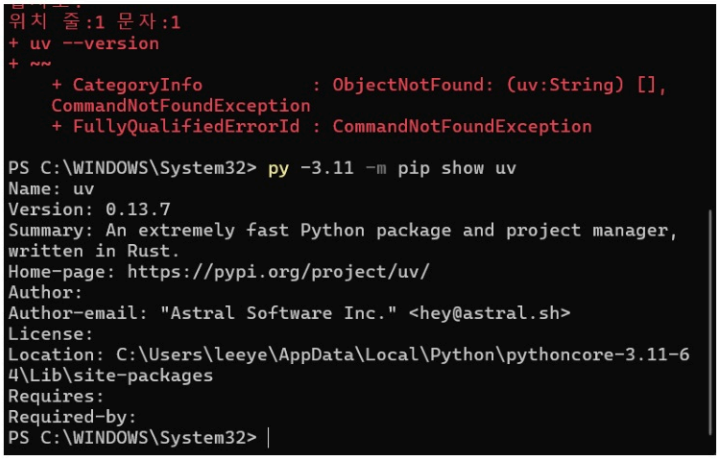
실행 화면 증거

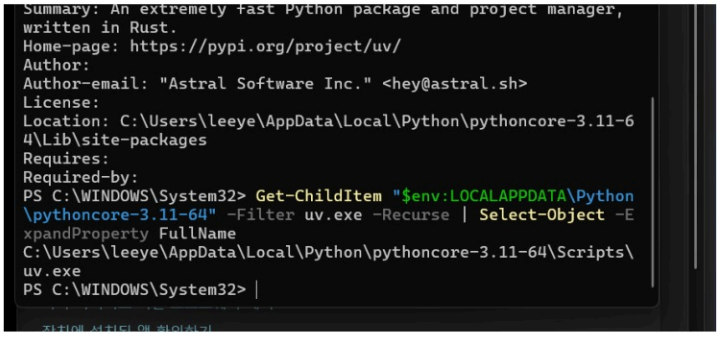
Python 설치와 버전 확인, uv 설치 및 PATH 수정, .venv 생성활성화, requirements 설치와 Jupyter Lab 실행 화면이다.

아래 캡처는 본인 Windows PowerShell 및 Jupyter Lab에서 실행한 화면이다. 코드와 출력, 오류 traceback을 보존했으며, 캡처 아래 설명은 해당 단계에서 확인한 결과와 원인을 요약한다.

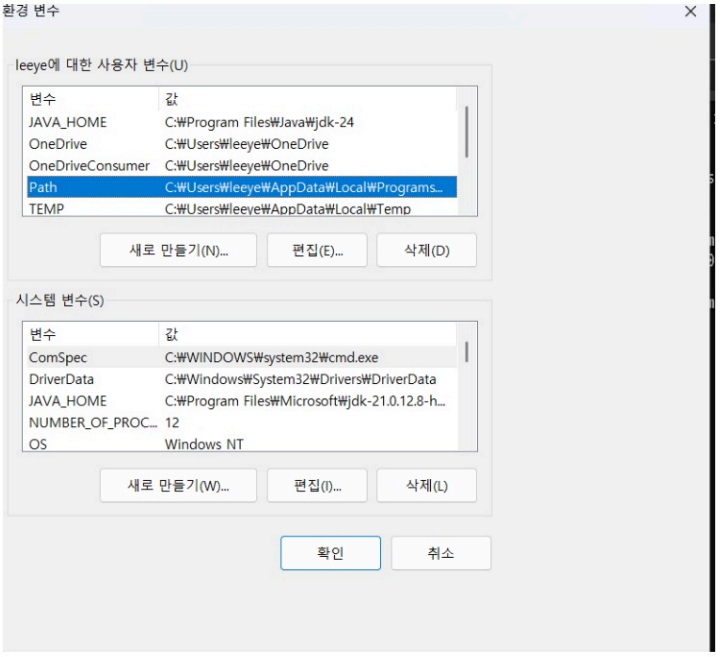


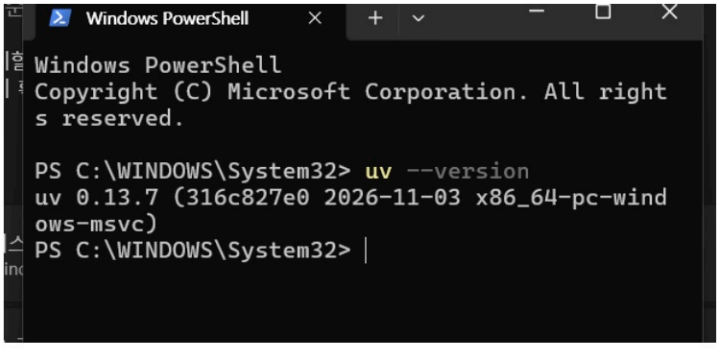
설치 확인 자료



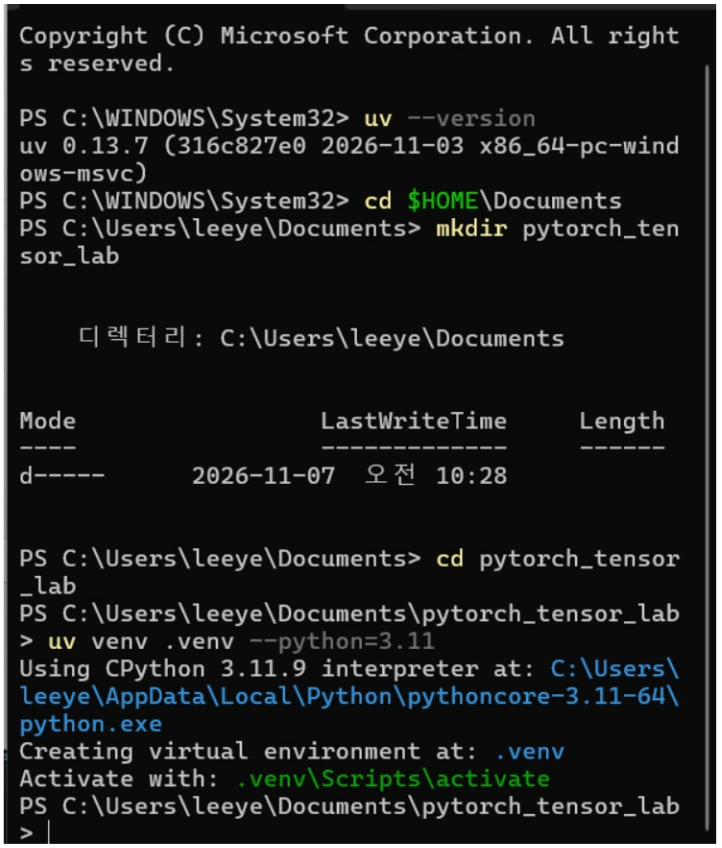


`uv.exe`가 Python 3.11의 `Scripts` 폴더에 설치된 것을 확인했다. 이 폴더를 사용자 PATH에 추가해 `uv` 명령어를 사용할 수 있도록 설정한다.

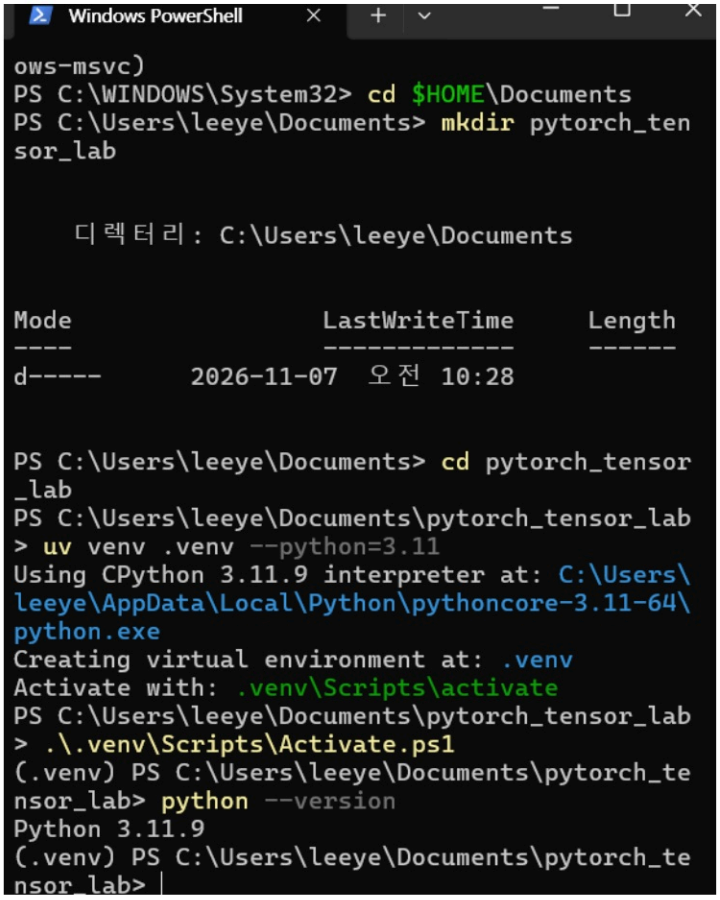




Python 3.11 환경에 설치한 uv 0.13.7을 PowerShell에서 정상적으로 실행하는 것을 확인했다.

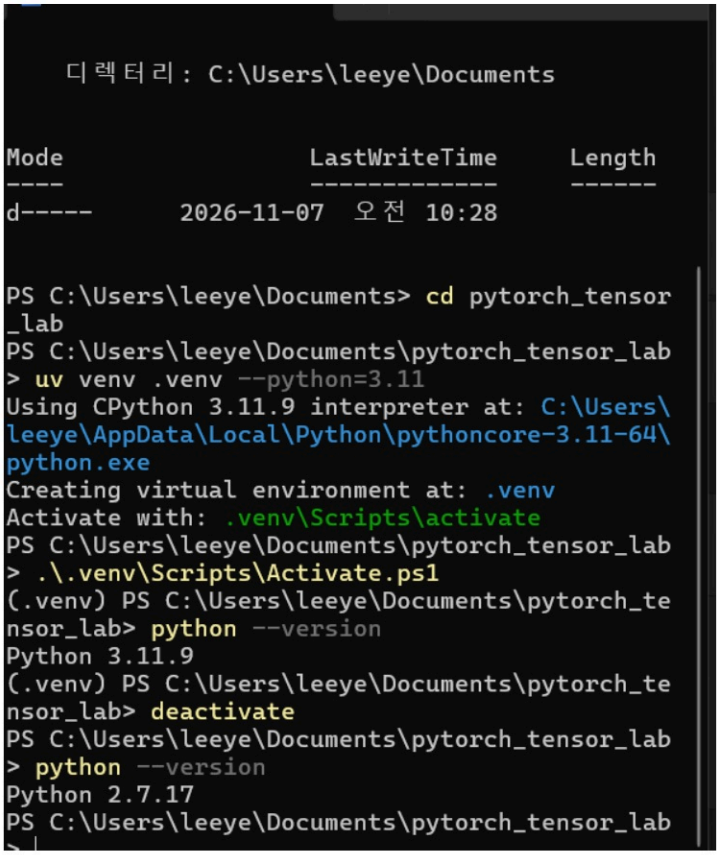


문서 폴더에 실습 프로젝트를 만들고, uv를 사용해 Python 3.11 가상환경을 생성했다.

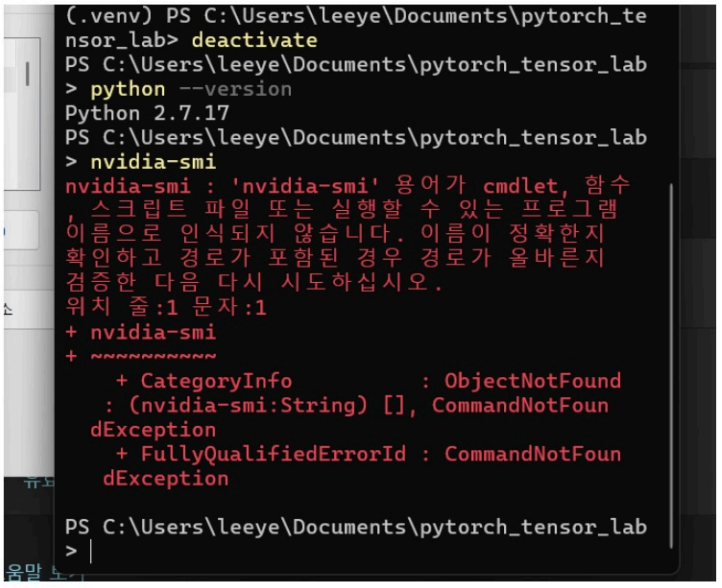


가상환경을 활성화한 뒤 Python 버전이 3.11.9로 적용된 것을 확인했다.

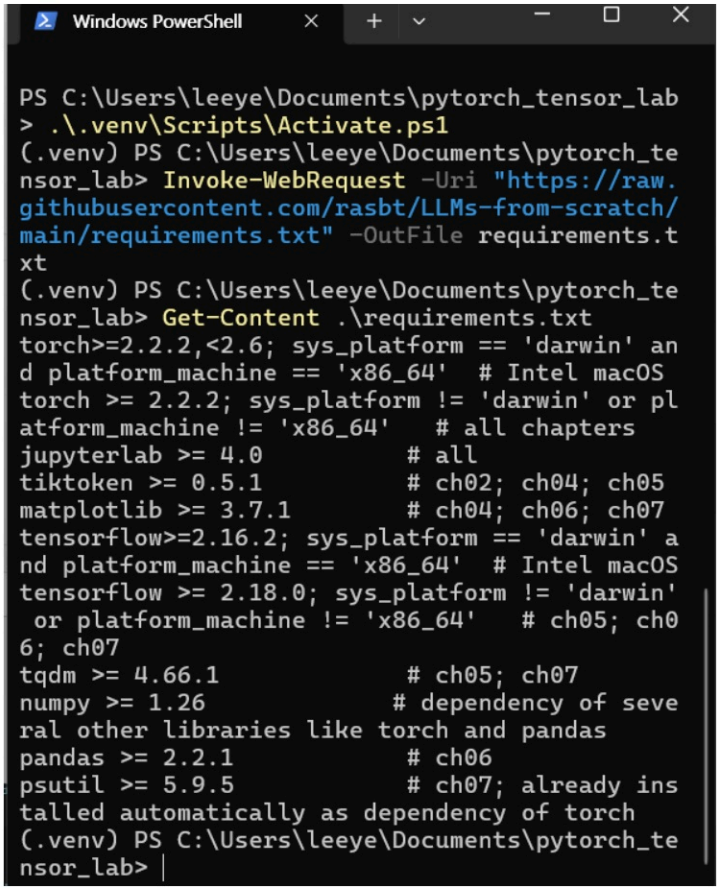
`pytorch_tensor_lab` 폴더에 Python 3.11.9 가상환경 `.venv`를 생성하고 활성화했다.



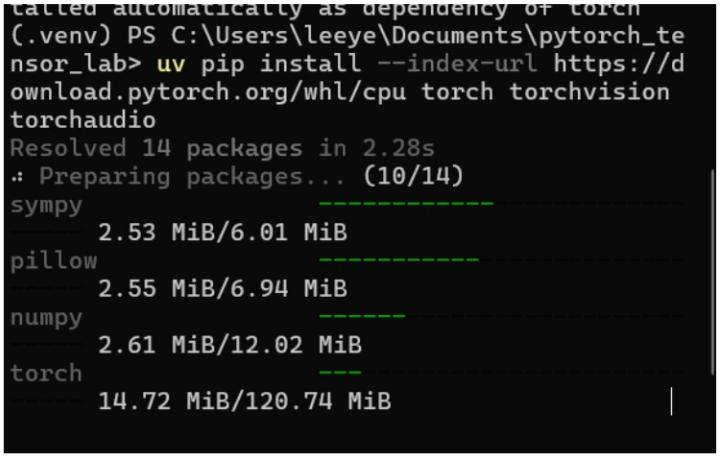
가상환경을 비활성화하자 시스템의 기본 Python 2.7.17로 돌아오는 것을 확인했다.



owerShell에서 `nvidia-smi` 명령을 찾지 못해 NVIDIA GPU 상태를 확인할 수 없었다. PyTorch 설치 후 CUDA 사용 여부를 확인할 예정이다.



LLMs-from-scratch의 `requirements.txt`를 프로젝트 폴더에 내려받아 의존성 목록을 확인했다



설치 중인 모습

Part 1. requirements.txt 전문

JupyterLab에서 실제로 확인한 프로젝트 파일 내용

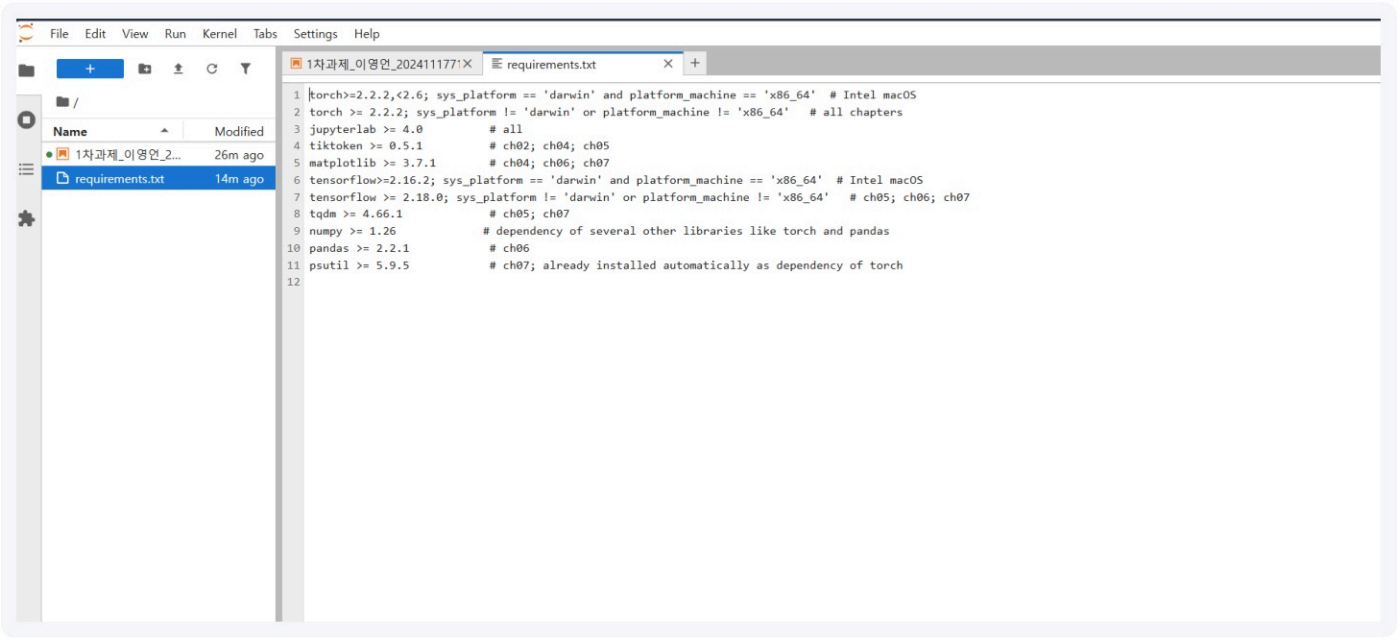
```
torch>=2.2.2,<2.6; sys_platform == 'darwin' and platform_machine == 'x86_64'  # Intel macOS
torch >= 2.2.2; sys_platform != 'darwin' or platform_machine != 'x86_64'  # all chapters
jupyterlab >= 4.0  # all
tiktoken >= 0.5.1  # ch02; ch04; ch05
matplotlib >= 3.7.1  # ch04; ch06; ch07
tensorflow>=2.16.2; sys_platform == 'darwin' and platform_machine == 'x86_64'  # Intel macOS
tensorflow >= 2.18.0; sys_platform != 'darwin' or platform_machine != 'x86_64'  # ch05; ch06; ch07
tqdm >= 4.66.1  # ch05; ch07
numpy >= 1.26  # dependency of several other libraries like torch and pandas
pandas >= 2.2.1  # ch06
psutil >= 5.9.5  # ch07; already installed automatically as dependency of torch
```

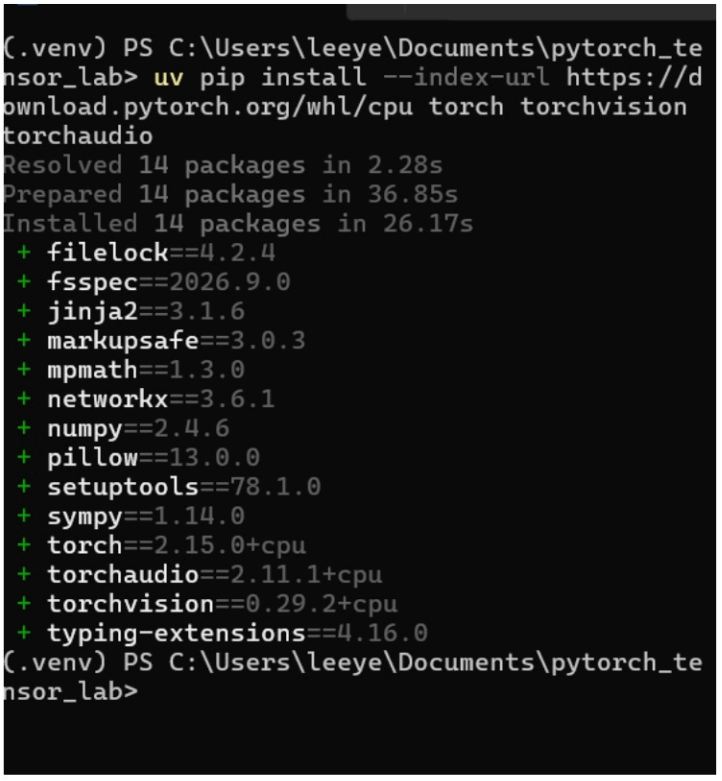
내용 해석

조건부 표기에서 darwin은 macOS를 가리킨다. 첫 Torch/TensorFlow 항목은 Intel macOS 전용이며, 다음 항목은 그 밖의 플랫폼용이다. requirements.txt에는 JupyterLab, tiktoken, matplotlib, TensorFlow, tqdm, NumPy, pandas, psutil도 포함되어 있다. Windows CPU 실습에서는 Torch 계열 CPU 빌드를 먼저 설치한 뒤 uv pip install -r requirements.txt를 실행했다.

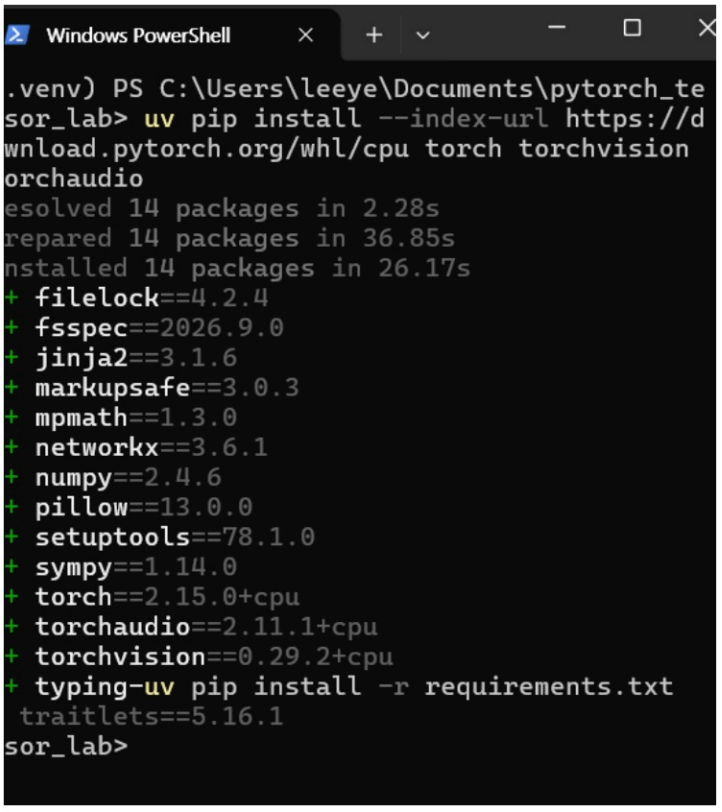
Part 1.4 실행 화면: requirements.txt 확인

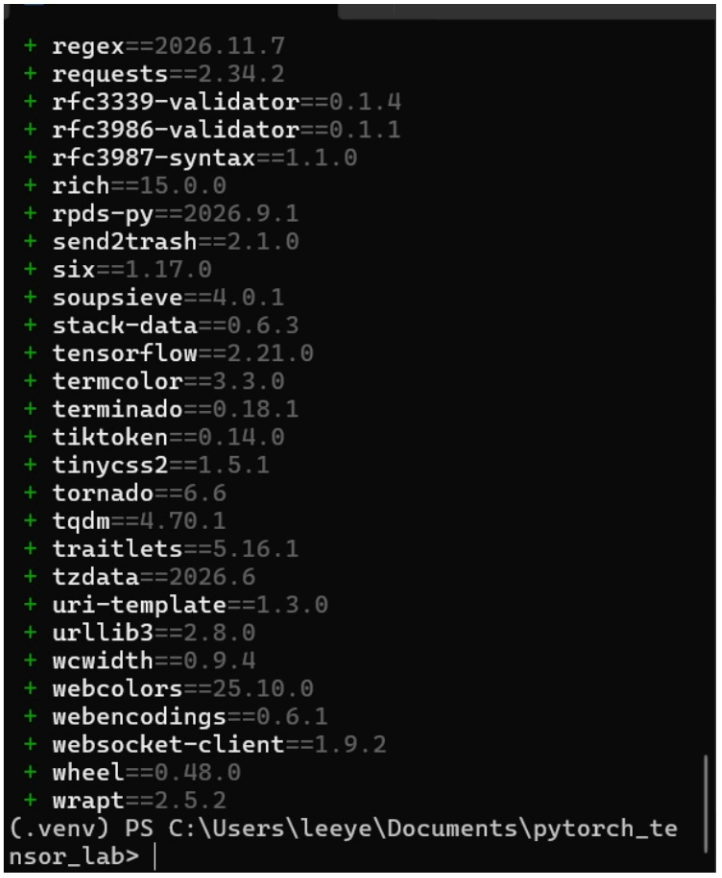
JupyterLab에서 프로젝트의 requirements.txt 전체 내용을 연 화면



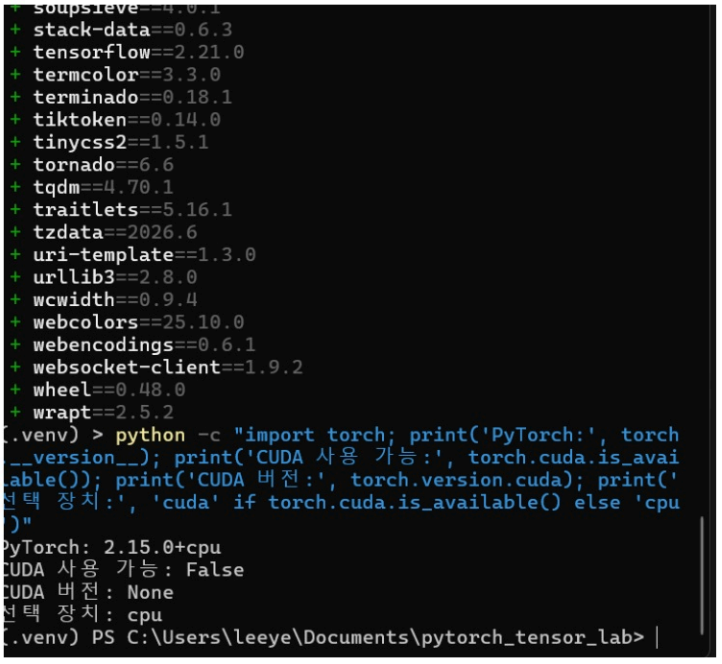


PyTorch, TorchVision, TorchAudio를 CPU 빌드로 설치했다.

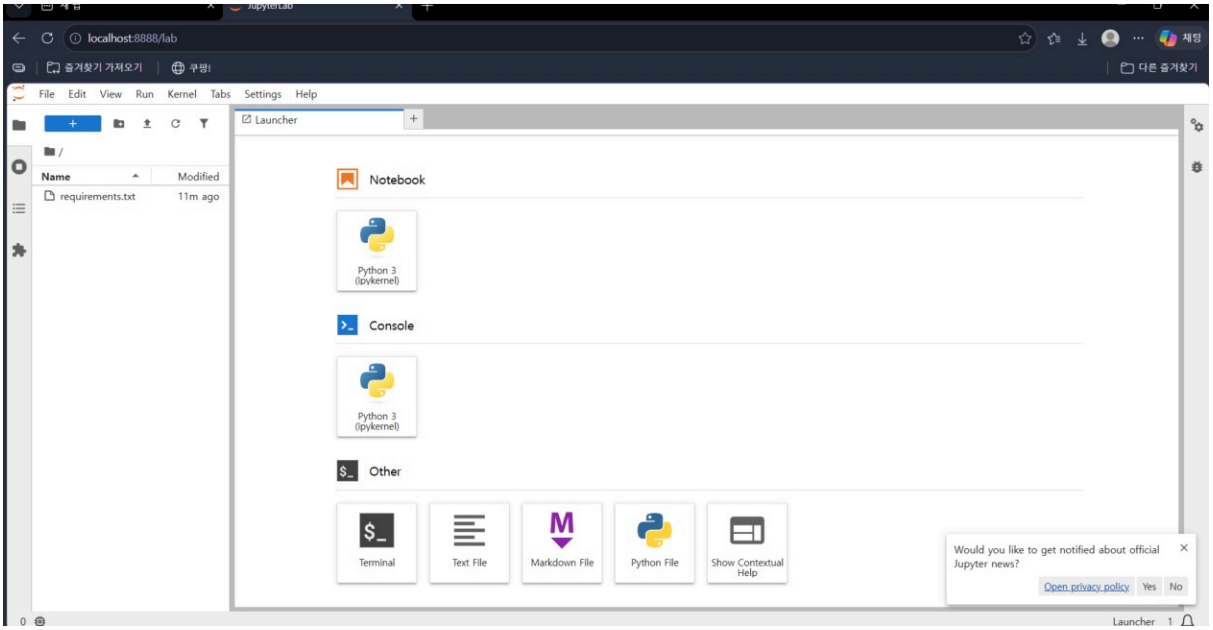




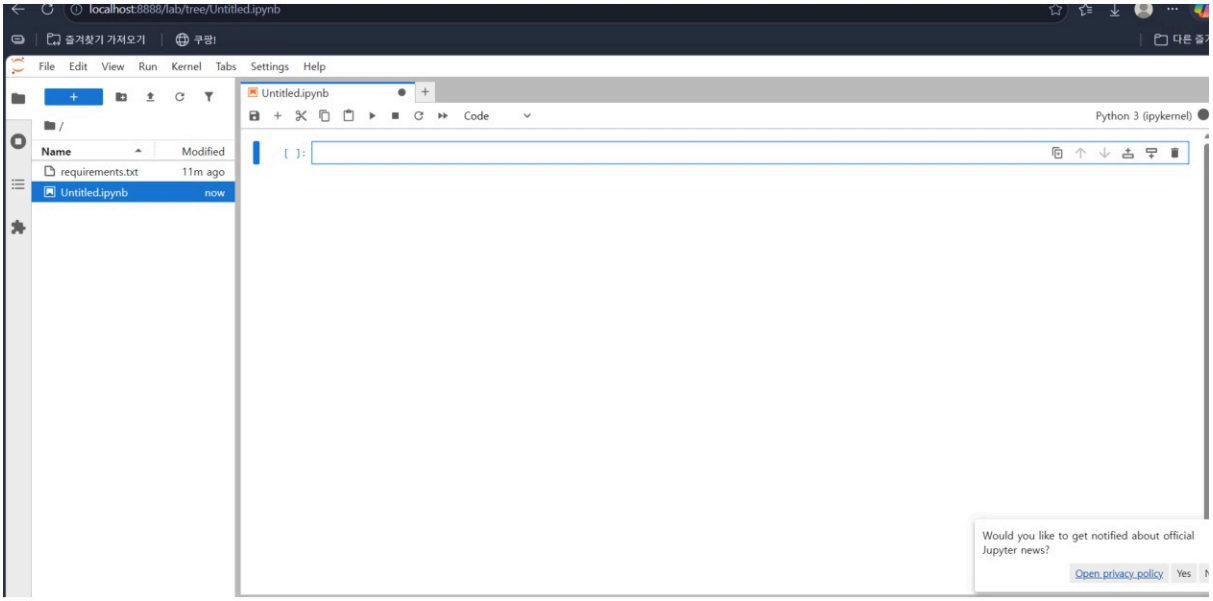
`requirements.txt`에 적힌 실습 의존성 패키지를 `.venv` 가상환경에 설치했다.



PyTorch 2.15.0 CPU 빌드를 설치했으며, CUDA 사용 가능 여부가 False로 확인되어 CPU 장치로 실습한다.



가상환경에서 `uv run jupyter lab`을 실행해 Jupyter Lab이 브라우저에서 열리는 것을 확인했다.



Jupyter Lab에서 Python 3 커널을 사용하는 새 실습 노트북을 만들었다.

Part 2. Tensor 생성과 속성

2.1 scalar, vector, matrix, 3차원 Tensor 및 dtype

2.1 차원별 Tensor 생성 결과

```
scalar = torch.tensor(7.0)
vector = torch.tensor([1, 2, 3])
matrix = torch.tensor([[1, 2, 3], [4, 5, 6]])
tensor_3d = torch.tensor([[[1, 2], [3, 4]],
                          [[5, 6], [7, 8]]])
```

scalar는 숫자 하나를 담는 0차원 Tensor이며 shape은 torch.Size([])다. vector는 값 3개를 한 축에 담아 shape (3,)이 된다. matrix는 두 행과 세 열이어서 shape (2,3)이다. 3차원 Tensor는 2개 묶음 안에 각각 2행, 2열이 들어 있는 구조라 shape (2,2,2)다. 중첩 대괄호의 각 단계가 축 하나에 대응하고, 각 축의 원소 개수가 shape의 각 숫자가 된다.

2.2 dtype이 float32와 int64로 나뉘는 이유

7.0처럼 소수점이 있는 Python 실수로 만든 scalar는 기본 부동소수점 dtype인 torch.float32가 되었다. 정수만 있는 리스트에서 만든 vector, matrix, 3차원 Tensor는 torch.int64가 되었다. 정수 연산은 소수점이 필요 없고, 실수 연산은 소수값을 표현하므로 값의 자료형에 따라 저장 형식과 메모리 사용이 달라진다. 필요하면 torch.tensor(..., dtype=torch.float32)처럼 dtype을 명시해 변환할 수 있다.

출력으로 확인할 속성

```
print(tensor)
print(tensor.shape)
print(tensor.ndim)
print(tensor.dtype)
```

출력된 실제 값은 Tensor의 내용, shape은 축별 크기, ndim은 축의 개수, dtype은 원소 저장 자료형을 나타낸다. 따라서 같은 숫자 배열이라도 shape과 dtype을 함께 확인해야 이후 연산에서 차원 불일치나 자료형 차이를 해석할 수 있다.

Part 2. Tensor 생성과 속성

2.3 생성 함수, 2.4 장치 확인과 CUDA 대체, 2.5 장치 혼합 연산

2.3 ones, full, randn의 차이

```
ones_tensor = torch.ones((3, 2))
full_tensor = torch.full((2, 3), -2)
random_tensor = torch.randn((2, 3))
```

torch.ones((3,2))는 3행 2열을 1로 채우며 기본 실수 dtype이므로 float32다. torch.full((2,3), - 2)는 지정한 shape에 - 2를 채우며 정수 값을 주어 int64가 출력됐다. torch.randn((2,3))은 평균 0, 표준편차 1인 정규분포에서 난수를 만들고 실수 dtype을 사용한다. shape을 바꾸면 행과 열 수가, full의 채움값을 바꾸면 모든 원소의 값이 바뀌며, randn은 실행할 때마다 난수 값이 달라질 수 있다.

2.4 device와 CUDA 환경 확인

```
print(t.device)
print(torch.cuda.is_available())
print(torch.version.cuda)
# CUDA 이름 조회는 사용 가능할 때만 실행
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
    moved = t.to("cuda")
else:
    moved = t.to("cpu")
```

캡처에서 CUDA 사용 가능 여부는 False, CUDA 버전은 None, 선택한 실습 장치는 cpu로 확인됐다. CPU 전용 PyTorch 빌드에는 CUDA 런타임이 포함되지 않으므로 CUDA 장치명도 조회할 수 없다. 이름 조회를 조건문으로 감싼 이유는 CUDA가 없는데 get_device_name(0)을 직접 호출하면 장치가 없다는 오류가 발생하기 때문이다. .to("cuda")도 이 환경에서는 실행할 수 없어 과제 지침대로 CPU 장치로 대체했다.

2.5 CPU Tensor와 CUDA Tensor의 연산

PyTorch는 같은 연산에 참여하는 Tensor들이 동일한 device에 있어야 한다. 따라서 CUDA가 있는 환경에서 CPU Tensor와 CUDA Tensor를 더하면 device mismatch 오류가 발생하고, 한쪽을 다른 쪽으로 .to(device) 이동하면 해결된다. 본 실습은 CUDA가 False라 두 장치를 동시에 만들 수 없어서 오류를 직접 재현하지 않았다. 대신 CUDA 불가 화면과 CPU 선택 코드를 제시했다. 이는 과제 유의사항의 GPU 미보유 대체 방법에 따른 처리다.

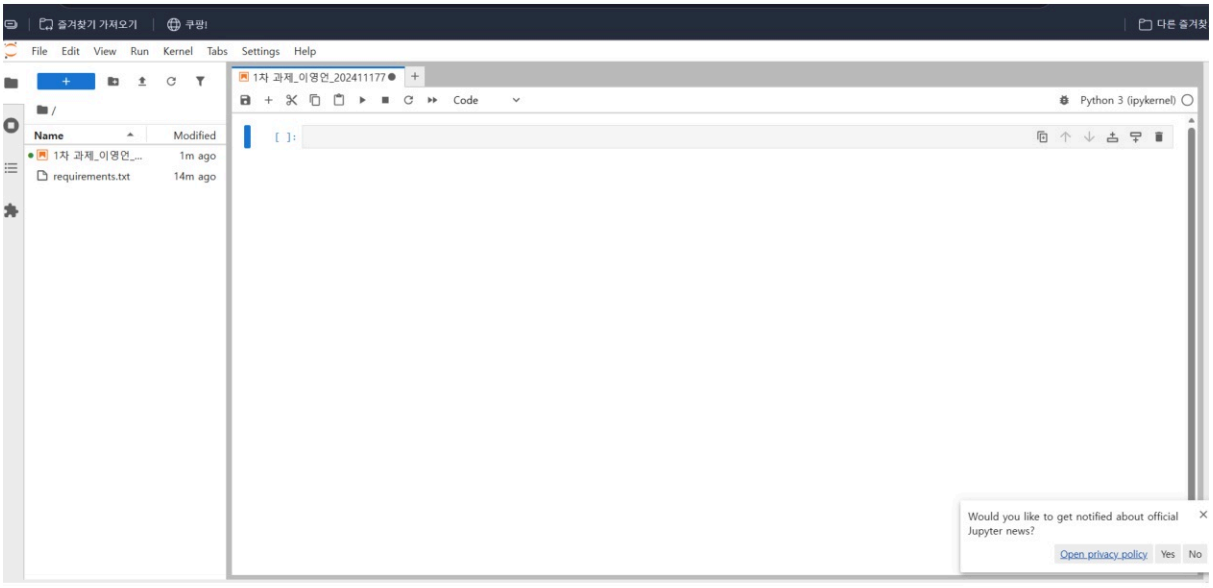
> CUDA 미사용은 실습 누락이 아니라, 확인된 장치 상태에 맞춘 과제 대체 수행이다. CPU에서 실행한 결과 shape과 계산은 그대로 확인했다.

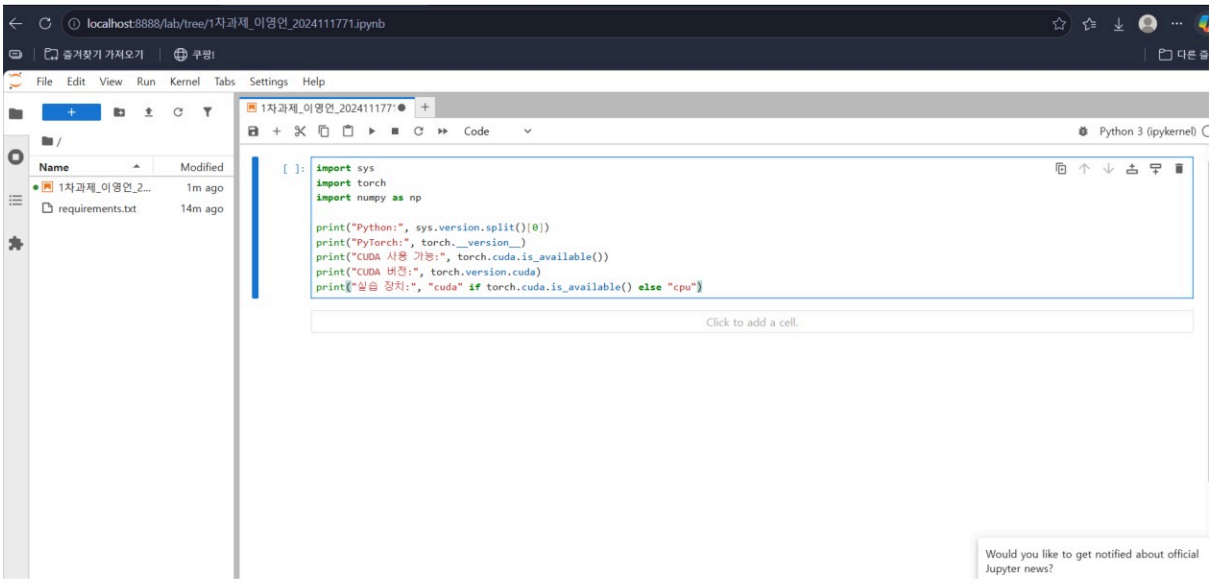
Part 2. Tensor 생성과 속성

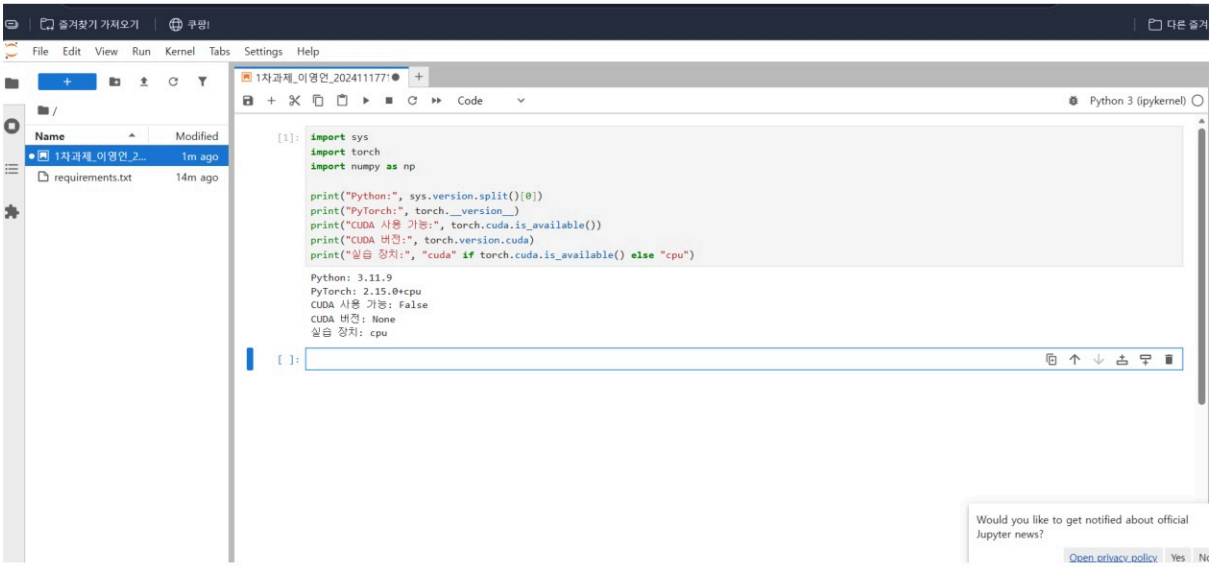
실행 화면 증거

차원별 Tensor와 dtype, 생성 함수, device/CUDA 확인, GPU가 없는 환경에서 적용한 CPU 대체 코드의 화면이다.

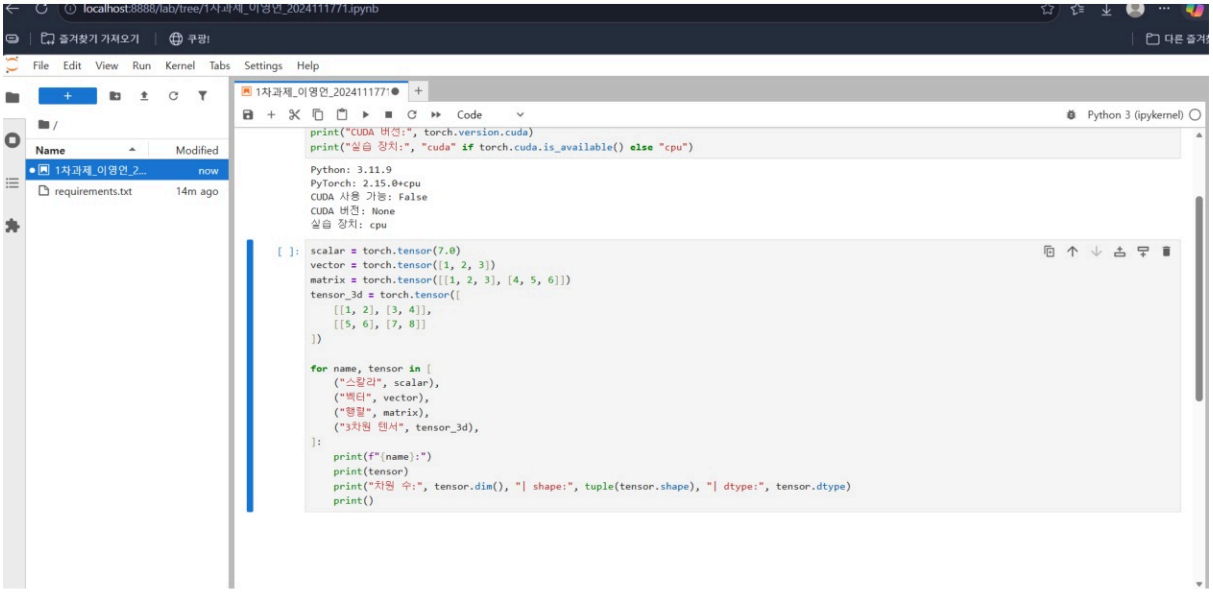
아래 캡처는 본인 Windows PowerShell 및 Jupyter Lab에서 실행한 화면이다. 코드와 출력, 오류 traceback을 보존했으며, 캡처 아래 설명은 해당 단계에서 확인한 결과와 원인을 요약한다.

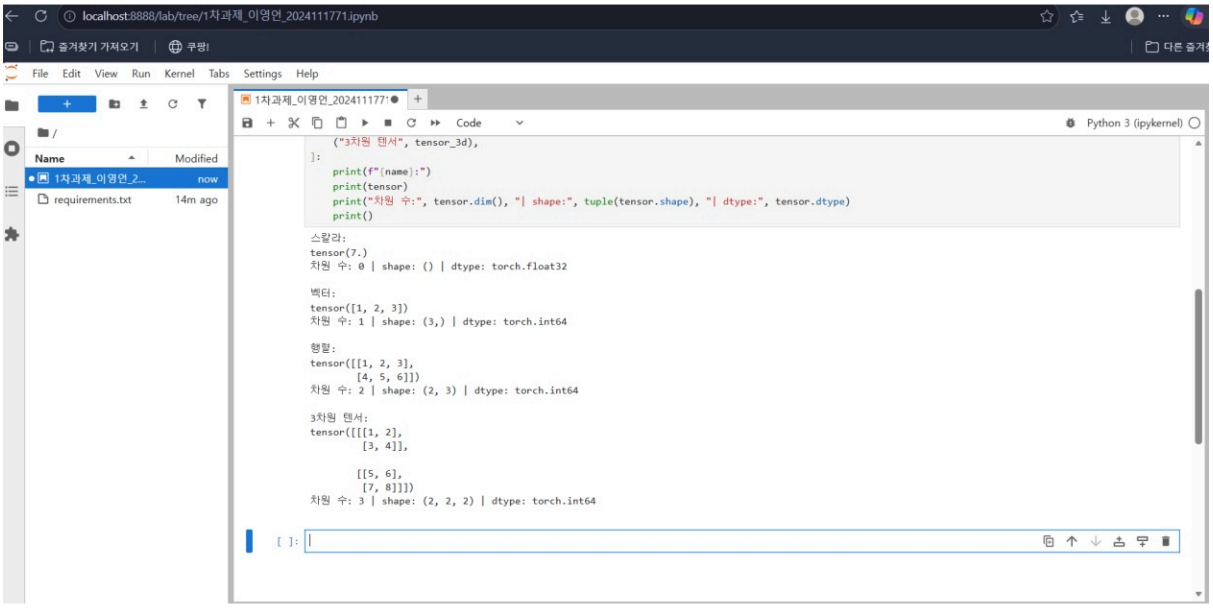

File Edit View Run Kernel Tabs Settings Help
1차 과제_이영언_202411177
Code
Python 3 (ipykernel)
Name
Modified
1차 과제_이영언_...
1m ago
requirements.txt
14m ago
[ ]:
Would you like to get notified about official Jupyter news?
Open privacy policy
Yes
No


localhost:8888/lab/tree/1차과제_이영언_2024111771.ipynb
File Edit View Run Kernel Tabs Settings Help
Code
Python 3 (ipykernel)
Name
Modified
1차과제_이영언_2...
1m ago
requirements.txt
14m ago
[ ]:
import sys
import torch
import numpy as np
print("Python:", sys.version.split()[0])
print("PyTorch:", torch.__version__)
print("CUDA 사용 가능:", torch.cuda.is_available())
print("CUDA 버전:", torch.version.cuda)
print("실습 장치:", "cuda" if torch.cuda.is_available() else "cpu")
Click to add a cell.
Would you like to get notified about official Jupyter news?

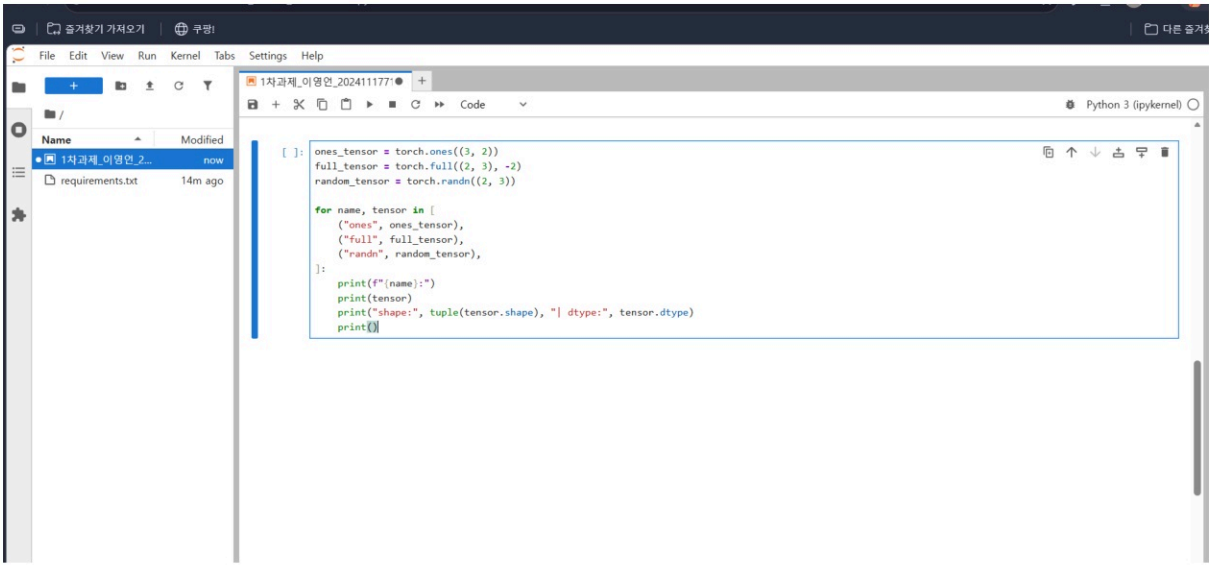

File Edit View Run Kernel Tabs Settings Help
Code
Python 3 (ipykernel)
Name
Modified
1차과제_이영언_2...
1m ago
requirements.txt
14m ago
[1]:
import sys
import torch
import numpy as np
print("Python:", sys.version.split()[0])
print("PyTorch:", torch.__version__)
print("CUDA 사용 가능:", torch.cuda.is_available())
print("CUDA 버전:", torch.version.cuda)
print("실습 장치:", "cuda" if torch.cuda.is_available() else "cpu")
Python: 3.11.9
PyTorch: 2.15.0+cpu
CUDA 사용 가능: False
CUDA 버전: None
실습 장치: cpu
[ ]:
Would you like to get notified about official Jupyter news?

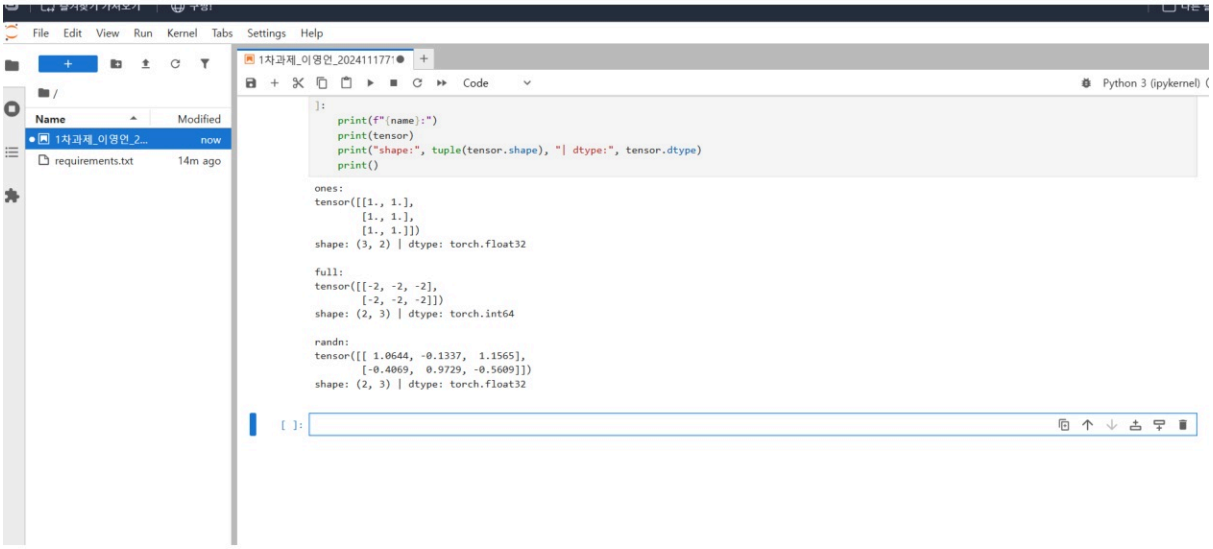
upyter 노트북에서 Python 3.11.9와 PyTorch CPU 빌드가 정상적으로 작동하며, CUDA를 사용할 수 없음을 확인했다.



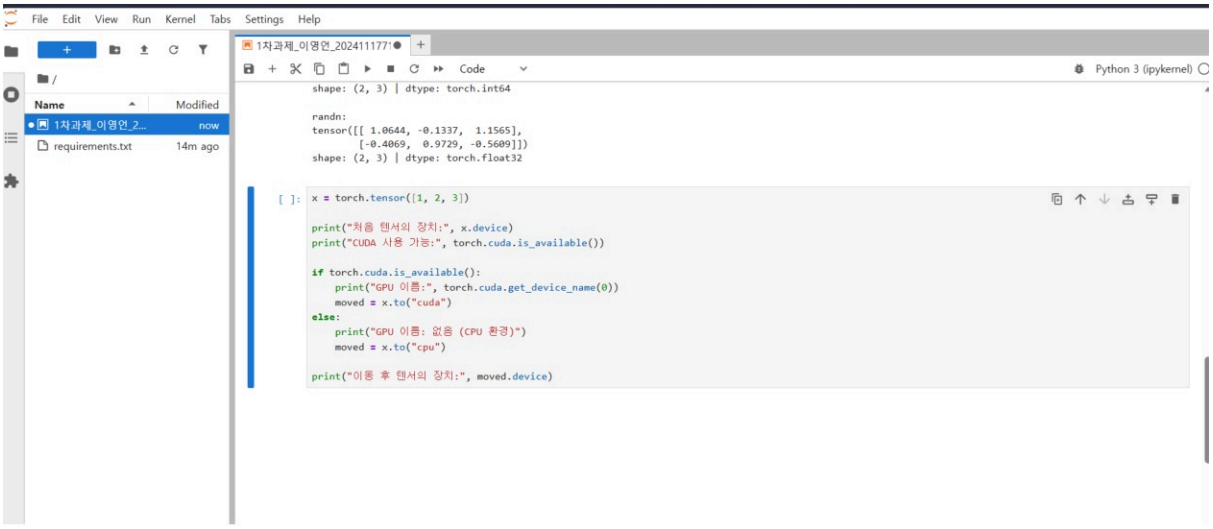


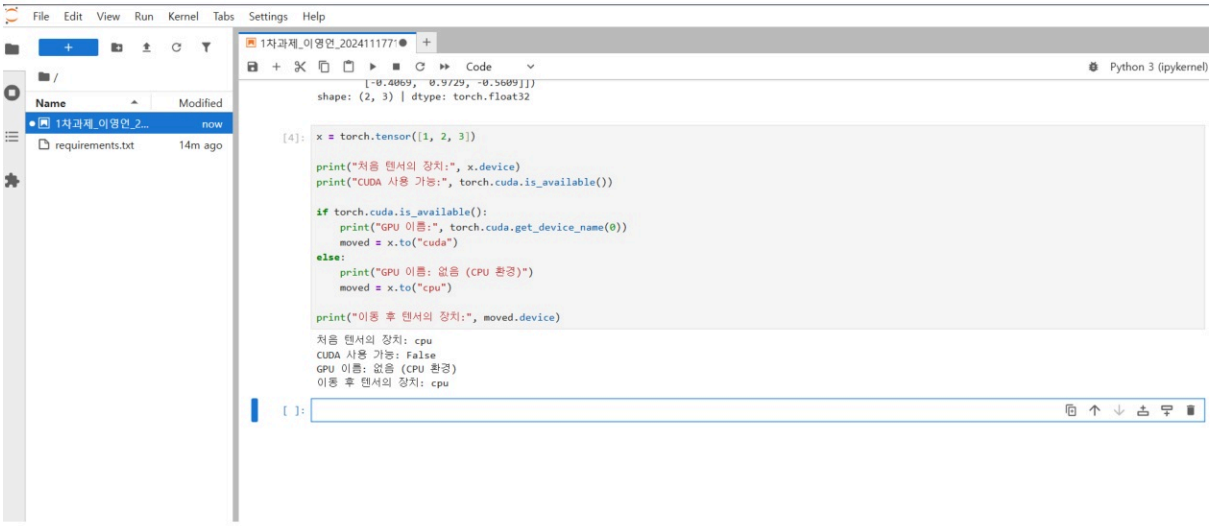
스칼라·벡터·행렬·3차원 텐서를 생성하고, 중첩된 대괄호와 차원 수, shape 및 자료형을 확인했다.



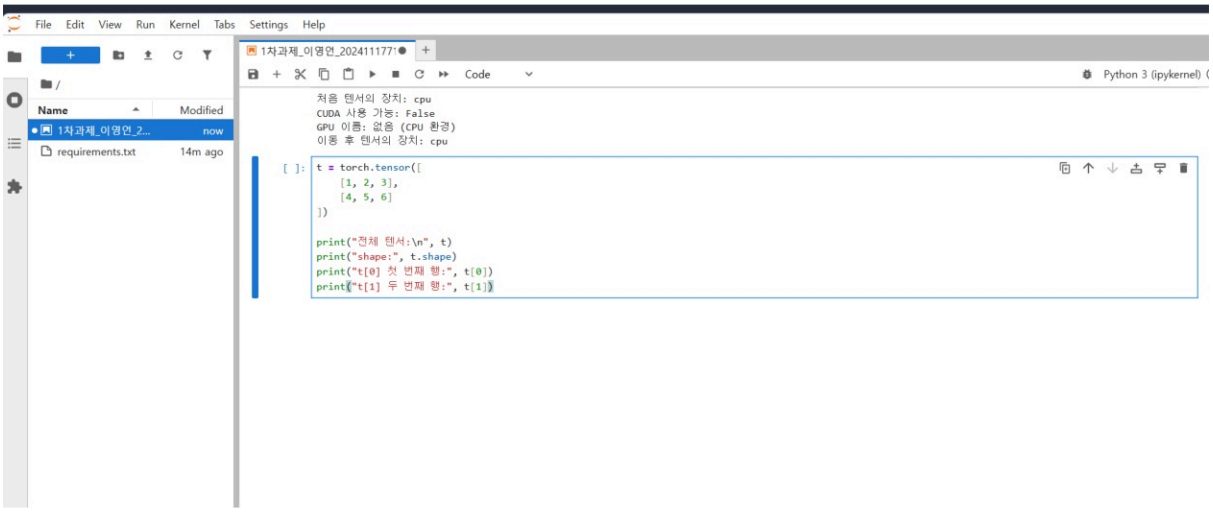


`torch.ones`, `torch.full`, `torch.randn`의 크기와 채울 값에 따라 텐서의 모양·값·자료형이 달라지는 것을 확인했다.





CUDA를 사용할 수 없어 텐서를 CPU에서 생성하고 CPU 장치에 유지했다. 이 환경에서는 CPU와 CUDA 텐서 간 연산 오류를 재현할 수 없으므로, CUDA 사용 불가 결과와 CPU 장치 선택으로 대체했다.



Part 3. 인덱싱과 슬라이싱

3.1 행 선택, 3.2 다양한 슬라이싱, 3.3 Ellipsis, 3.4 Boolean mask와 오류

3.1 2행 3열 Tensor와 행 인덱싱

```
t = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])
print(t.shape)  # torch.Size([2, 3])
print(t[0])     # tensor([1, 2, 3])
print(t[1])     # tensor([4, 5, 6])
```

첫 번째 축은 행, 두 번째 축은 열이다. t[0]은 0번 행 전체, t[1]은 1번 행 전체를 고른다. Python 인덱스는 0부터 시작하므로 화면의 두 행에 각각 인덱스 0과 1이 대응한다.

3.2 슬라이싱 결과를 행과 열로 해석

```
t[:, 1:]   # 모든 행, 1번 열부터 끝까지
t[1:, :2]  # 1번 행부터, 0~1번 열
t[::2]     # 행을 2칸 간격으로 선택
t[::3]     # 행을 3칸 간격으로 선택
t[-1]      # 마지막 행
t[:, -2]   # 모든 행의 끝에서 두 번째 열
```

t[:,1:]의 결과는 [[2,3],[5,6]]이고 첫 열을 제외한다. t[1:,:2]는 [[4,5]]로 두 번째 행에서 앞의 두 열을 가져온다. 행 축 길이가 2이므로 t[::2]와 t[::3]은 모두 0번 행 하나만 선택한다. t[-1]은 마지막 행 [4,5,6], t[:,-2]는 끝에서 두 번째 열의 값 [2,5]다. 음수 인덱스는 뒤에서부터 세며 -1이 마지막 원소다.

범위를 벗어난 인덱스 t[-5]

행이 2개인 Tensor에서 허용되는 행 인덱스는 0, 1 또는 음수 -2, -1이다. t[-5]는 첫 번째 축의 범위를 벗어나므로 IndexError가 발생했다. 오류 원인은 음수 인덱스 자체가 아니라 행 개수보다 큰 음수 오프셋을 사용한 것이다. 마지막 행을 얻으려면 t[-1]을 사용한다.

3.3 4차원 Tensor와 Ellipsis (...)

```
x = torch.arange(1, 17).reshape(2, 2, 2, 2)  # 캡처 고정값
x[1, ..., 1]; x[0, ..., 0]
x_random = torch.randn(2, 2, 2, 2)  # 과제 지정 코드
print(x_random.shape, x_random[1, ..., 1].shape, x_random[0, ..., 0].shape)
```

캡처에는 arange 고정값 결과가 있고, 과제 지정 randn 코드도 함께 실었다. 난수 값은 실행마다 다르지만 전체 shape은 (2,2,2,2), 두 인덱싱 결과는 (2,2)다. Ellipsis(...)는 생략된 축을 모두 선택한다. 고정값 결과는 [[10,12],[14,16]]과 [[1,3],[5,7]]이다.

3.4 Boolean mask: 추출, 오류, 수정

```
v = torch.tensor([1, 2, 3, 4, 5, 6])
v[v > 3]       # tensor([4, 5, 6])
v[v % 2 == 0]  # tensor([2, 4, 6])
v[v % 2 == 1]  # tensor([1, 3, 5])
wrong_mask = torch.tensor([True, False, True])
t[wrong_mask]  # IndexError
row_mask = torch.tensor([True, False])
t[row_mask]    # tensor([[1, 2, 3]])
```

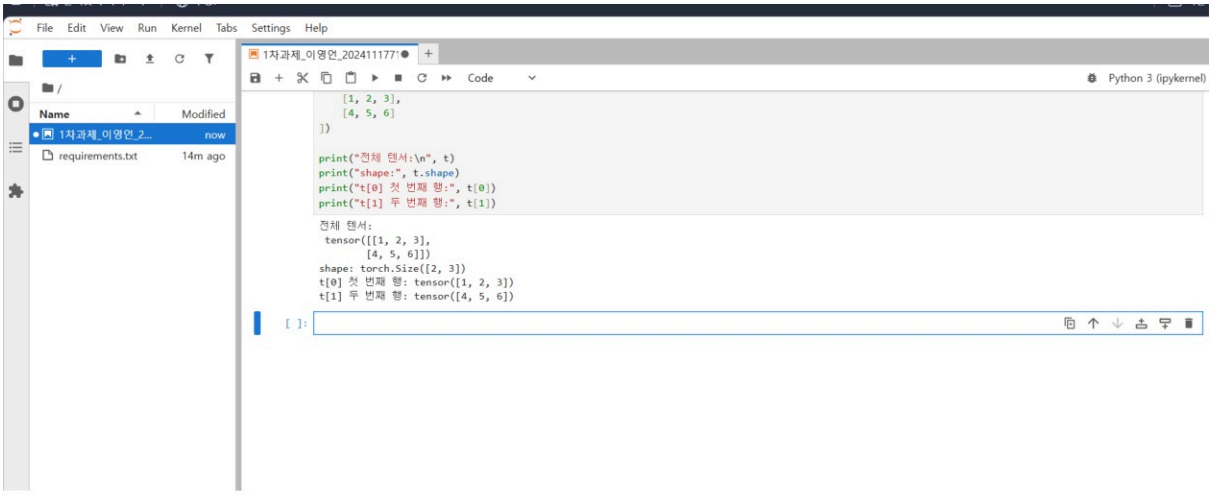
조건식은 원본 Tensor와 같은 위치별 True/False 마스크를 만든다. 1차원 v에서는 마스크 길이 6이 각 원소와 대응한다. 2행 Tensor t에 길이 3인 wrong_mask를 첫 번째 축에 적용하면 mask 길이 3과 행 길이 2가 달라 IndexError가 발생한다. 각 행을 선택할 마스크는 길이가 행 수와 같은 2여야 하므로 [True,False]로 수정하면 첫 번째 행만 반환된다.

Part 3. 인덱싱과 슬라이싱

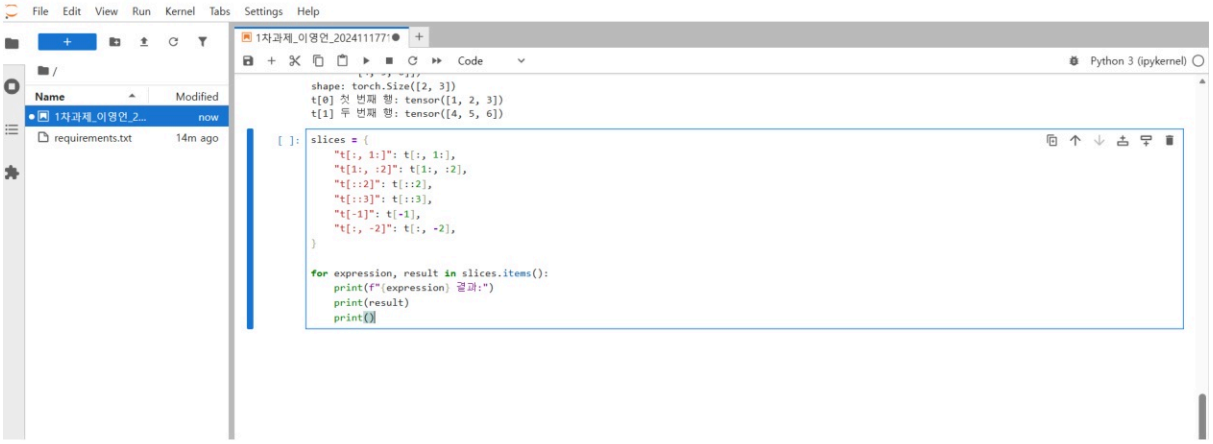
실행 화면 증거

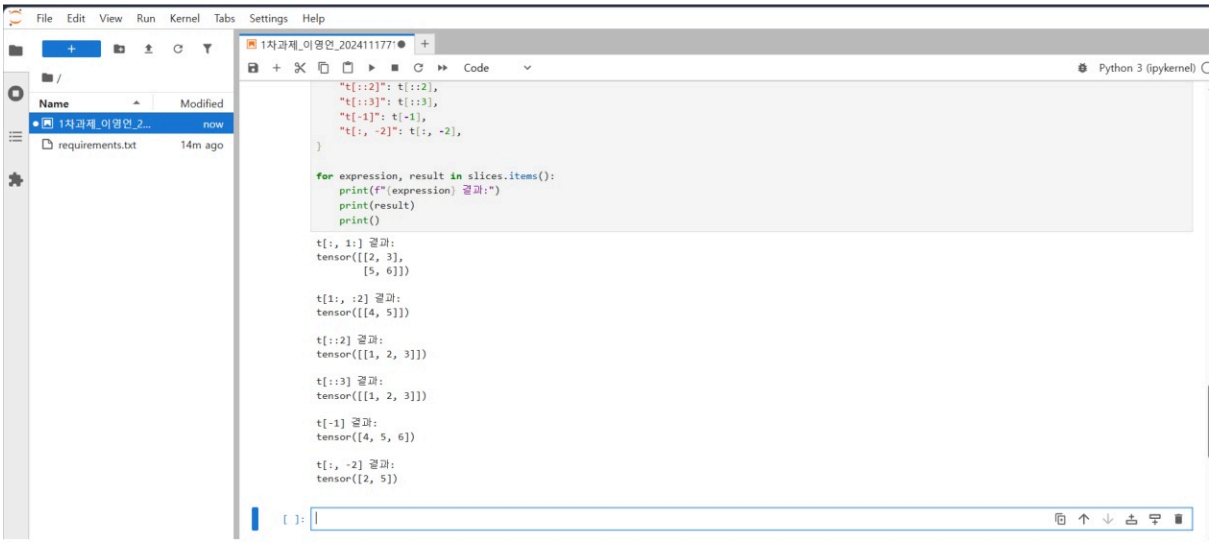
행 선택, 슬라이싱, 범위 오류, Ellipsis, boolean mask 오류와 길이를 맞춘 수정 결과 화면이다.

아래 캡처는 본인 Windows PowerShell 및 Jupyter Lab에서 실행한 화면이다. 코드와 출력, 오류 traceback을 보존했으며, 캡처 아래 설명은 해당 단계에서 확인한 결과와 원인을 요약한다.

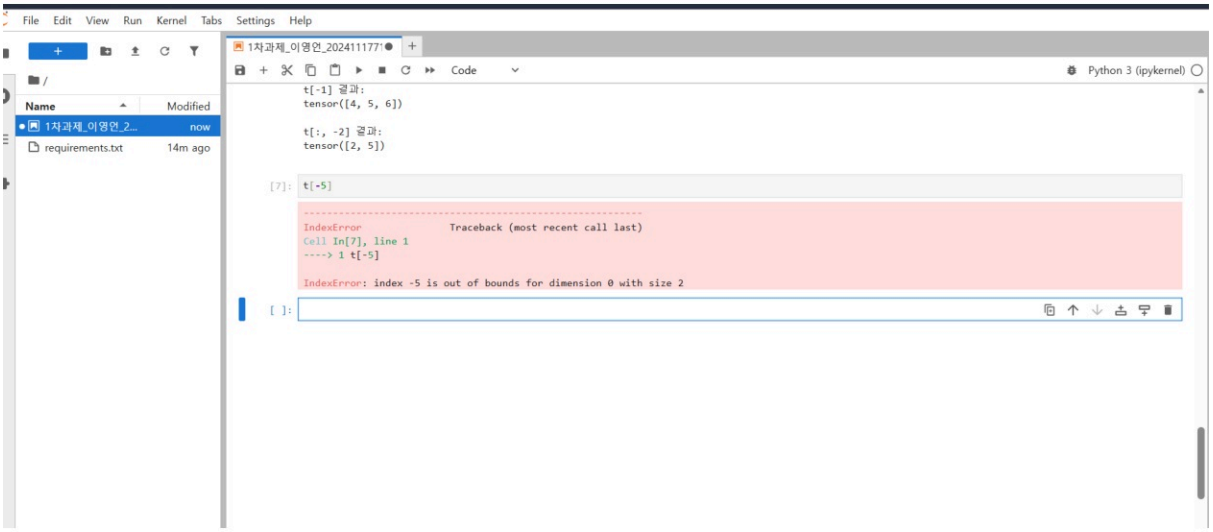


2행 3열 텐서의 shape을 확인하고, `t[0]`과 `t[1]`로 각 행을 선택했다.

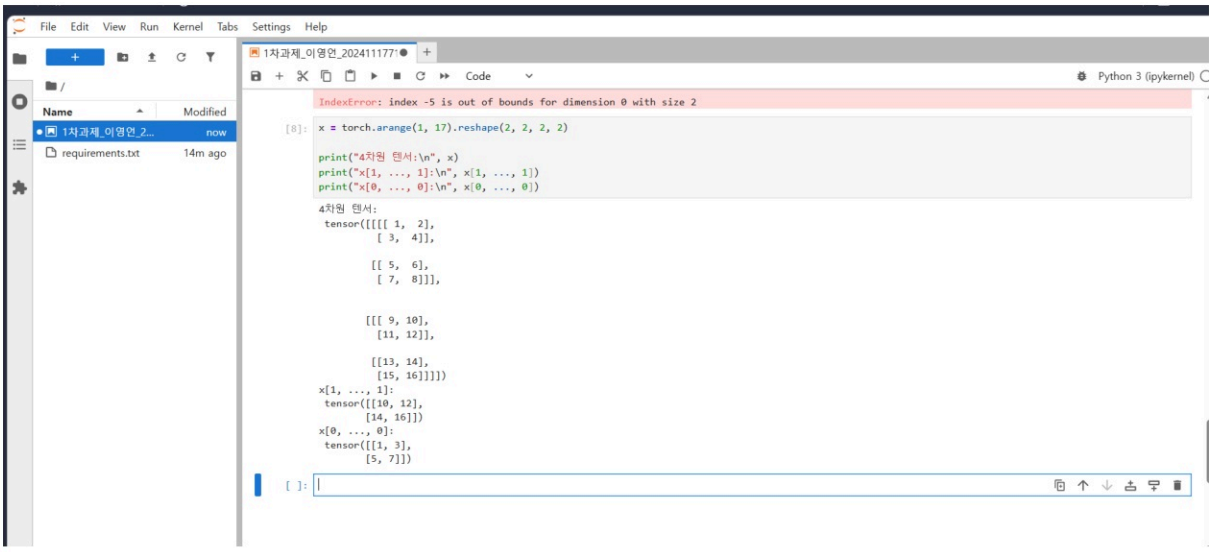




2×3 텐서에서 슬라이싱과 음수 인덱싱으로 원하는 행과 열을 선택한 결과를 확인했다.



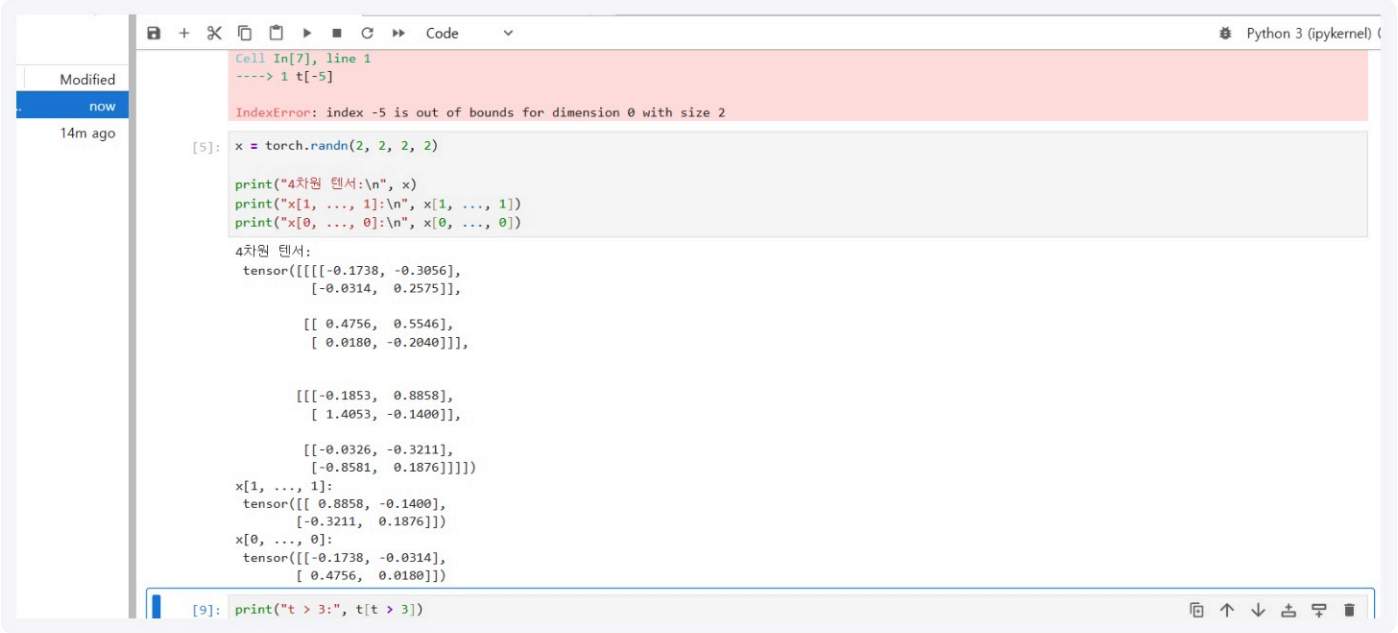
행이 2개인 텐서에서 범위를 벗어난 인덱스 `-5`를 사용해 `IndexError`가 발생했다. 사용할 수 있는 행 인덱스는 `-2`와 `-1`이다.



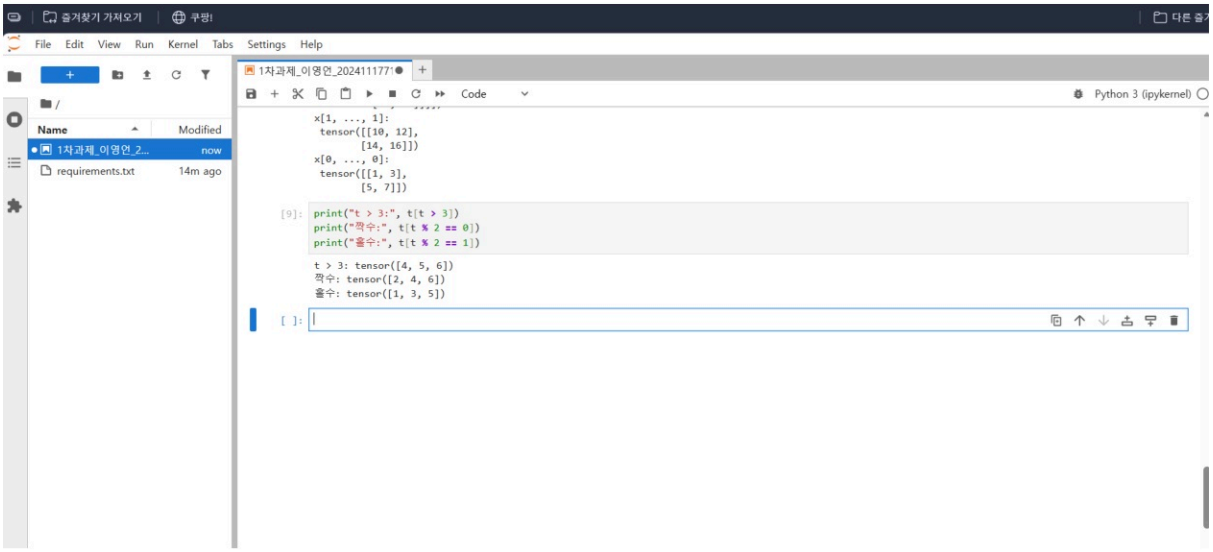
4차원 텐서에서 줄임표(`...`)로 중간 차원을 생략해 원하는 값을 선택했다.

Part 3.3 실행 화면: 4차원 Tensor와 Ellipsis

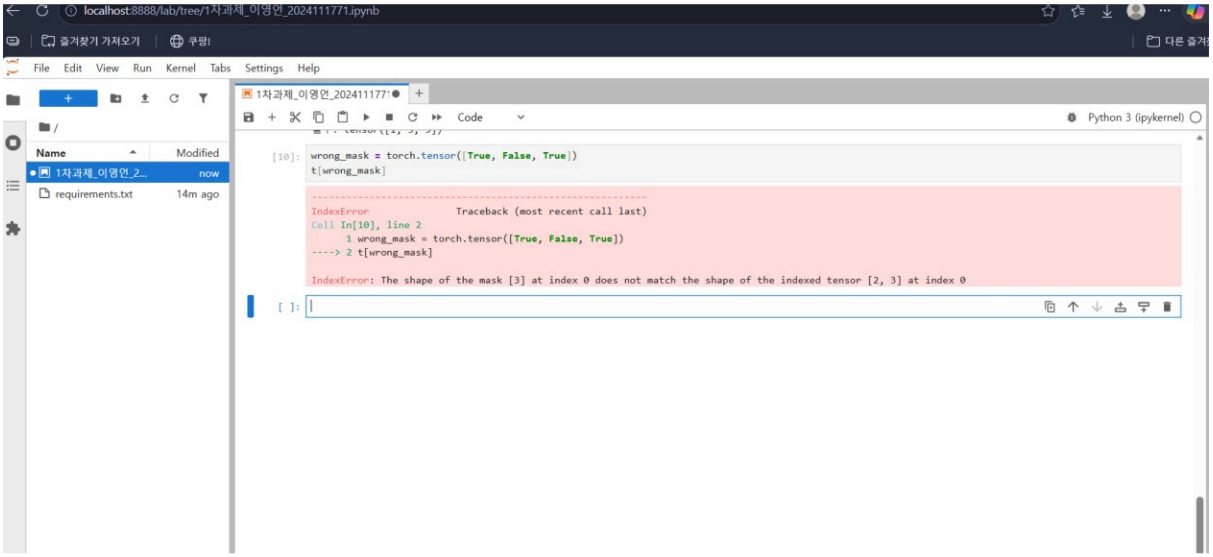
torch.randn(2, 2, 2, 2), x[1, ..., 1], x[0, ..., 0] 실행 결과



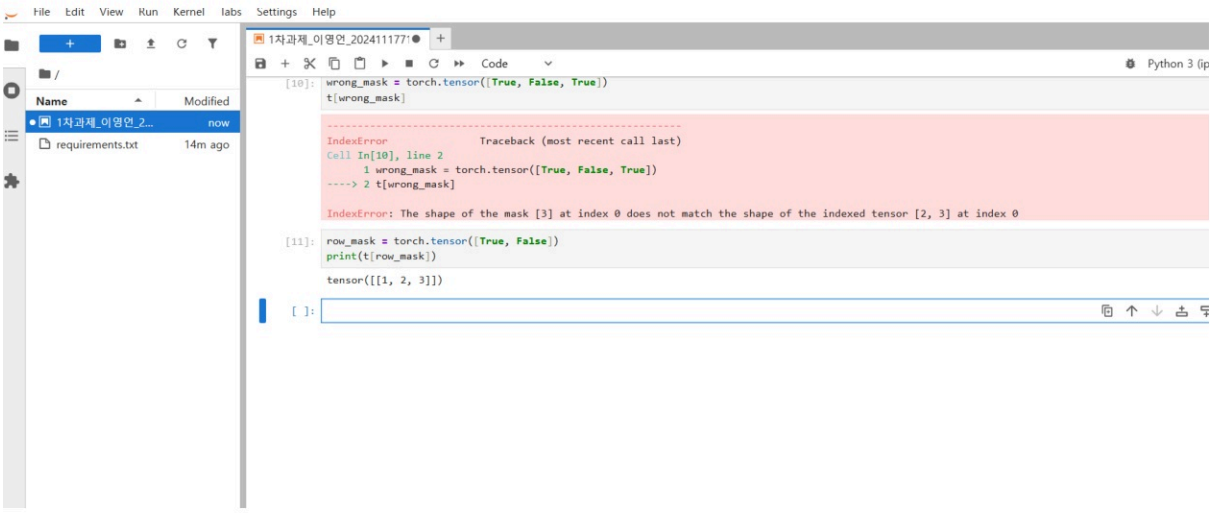
화면 위의 t[-5] IndexError는 앞 단계에서 의도적으로 재현한 오류다.



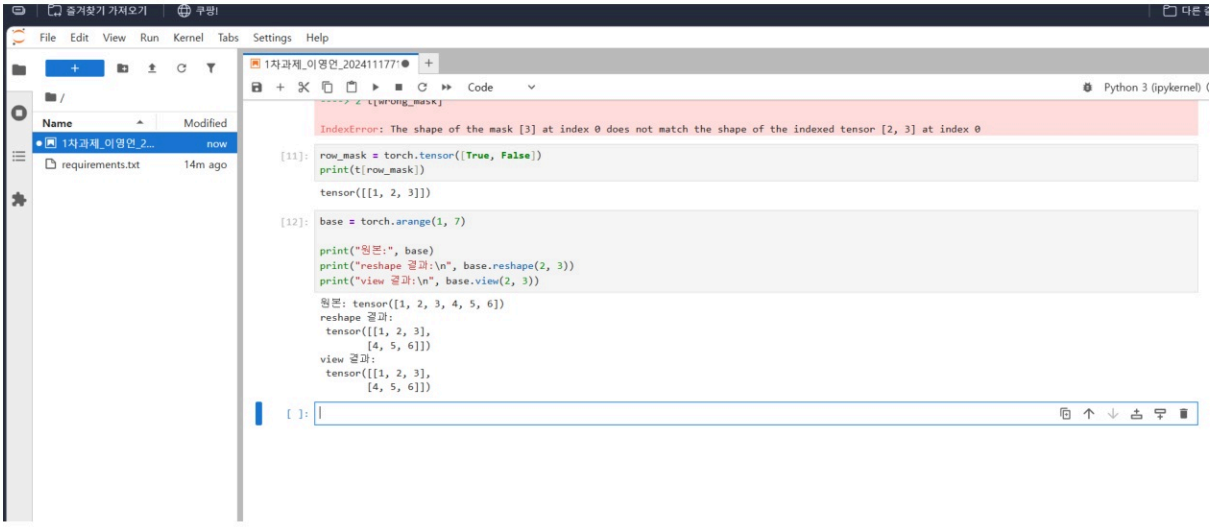
조건식으로 텐서에서 3보다 큰 값과 짝수·홀수 값을 각각 선택했다.



텐서의 행은 2개인데 길이가 3인 불리언 마스크를 적용해 `IndexError`가 발생했다.



텐서의 행 수와 길이를 맞춘 불리언 마스크로 첫 번째 행을 선택했다.



1차원 텐서를 `reshape()`와 `view()`로 2×3 형태로 바꾼 결과를 비교했다.

Part 4. Tensor 구조 변경

4.1 reshape와 view, 4.2 flatten과 -1, 4.3 unsqueeze와 squeeze

4.1 torch.arange(12), reshape(3,4), view(3,4)

```
base12 = torch.arange(12)
a = base12.reshape(3, 4)
b = base12.view(3, 4)
```

torch.arange(12)는 0부터 11까지 값 12개를 만든다. reshape(3,4)와 view(3,4)는 원소 수를 유지한 채 이를 3행 4열로 재배열한다. 두 결과의 shape은 모두 (3,4)이고 값의 순서는 행 우선으로 이어진다. view는 현재 메모리 배치가 연속적이어야 새 shape을 만들 수 있다. reshape은 가능하면 view처럼 동작하지만, 메모리 배치가 맞지 않으면 새 메모리를 복사해 결과를 만들 수 있다. 따라서 결과 모양은 같아도 메모리 공유가 항상 같다고 단정하면 안 된다.

4.2 평면화와 reshape(-2) 오류

```
base = torch.arange(1, 7)
matrix = base.reshape(2, 3)
matrix.flatten()  # tensor([1, 2, 3, 4, 5, 6])
base.reshape(-1)  # 크기를 자동 계산한 1차원 Tensor
base.reshape(-2)  # RuntimeError
```

flatten()은 모든 원소를 1차원으로 펼친다. reshape(-1)은 전체 원소 수와 다른 차원 크기를 보고 해당 축의 크기를 자동 계산하라는 뜻이다. 반면 -2는 PyTorch reshape에서 자동 추론을 뜻하지 않는 잘못된 크기이므로 invalid shape dimension RuntimeError가 발생했다. 해결은 -2를 허용되는 자동 추론 표기 -1로 바꾸는 것이다.

4.3 unsqueeze와 squeeze로 축 추가제거

```
matrix.shape                  # (2, 3)
matrix.unsqueeze(0).shape     # (1, 2, 3)
matrix.unsqueeze(1).shape     # (2, 1, 3)
matrix.unsqueeze(-2).shape    # (2, 1, 3)
matrix.unsqueeze(1).squeeze().shape  # (2, 3)
base12.unsqueeze(2)  # IndexError
```

unsqueeze는 원소를 추가하지 않고 길이 1인 축만 끼워 넣는다. 축 번호 0은 맨 앞, 1은 두 번째 위치, -2는 뒤에서 두 번째 위치다. 따라서 (2,3)에 unsqueeze(0)을 적용하면 (1,2,3), unsqueeze(1) 또는 unsqueeze(-2)를 적용하면 (2,1,3)이 된다. squeeze()는 크기가 1인 축을 제거하여 원래 shape으로 되돌린다. 1차원 Tensor의 허용 위치는 [-2,1]인데 unsqueeze(2)는 이 범위를 벗어나 IndexError가 발생했다.

Part 4. Tensor 구조 변경

4.4 전치와 permute, 4.5 expand와 repeat, 4.6 resize_ 확장

4.4 mT와 permute(0, 2, 1)

```
m = torch.arange(12).reshape(3, 4)
m.T.shape  # (4, 3)
x = torch.randn(3, 4, 5)
x.permute(0, 2, 1).shape  # (3, 5, 4)
```

2차원 m.T는 행과 열 축을 맞바꾸므로 (3,4)가 (4,3)이 된다. permute의 각 인자는 새 결과의 해당 위치에 놓일 원래 축 번호다. (0,2,1)은 원래 0번 축을 첫 번째로 유지하고, 원래 2번 축을 두 번째로, 원래 1번 축을 세 번째로 옮긴다. 따라서 입력 (3,4,5)는 (3,5,4)가 된다. permute는 데이터 축의 순서를 바꾸는 연산이며, 값을 새로 정렬하는 것과 shape 축 배치 변경을 구분해야 한다.

4.5 expand와 repeat의 메모리 차이

```
row = torch.tensor([[1, 2, 3]])
expanded = row.expand(2, 3)
repeated = row.repeat(2, 1)
row.data_ptr() == expanded.data_ptr()  # True
row.data_ptr() == repeated.data_ptr()  # False
```

expand와 repeat의 출력값은 모두 [[1,2,3],[1,2,3]]처럼 보인다. expand는 크기 1인 축을 stride 방식으로 확장해 원본 저장공간을 공유하므로 data_ptr이 같다. 같은 값을 여러 번 실제 복사하지 않아 메모리 사용이 작다. repeat은 값을 새 저장공간에 복제하므로 data_ptr이 다르고 반복 수에 비례해 메모리를 더 쓴다. expand 결과의 반복 위치는 같은 원본 메모리를 가리키므로 그 위치를 각각 독립적으로 수정할 수 있다고 생각하면 안 된다.

4.6 resize_, unsqueeze, expand 순서

```
a = torch.arange(1, 9).resize_(2, 4)
a.unsqueeze(1).shape       # (2, 1, 4)
a.unsqueeze(1).expand(2, 2, 4).shape  # (2, 2, 4)
b = torch.arange(1, 13).resize_(3, 4)
b.unsqueeze(1).expand(3, 5, 4).shape  # (3, 5, 4)
```

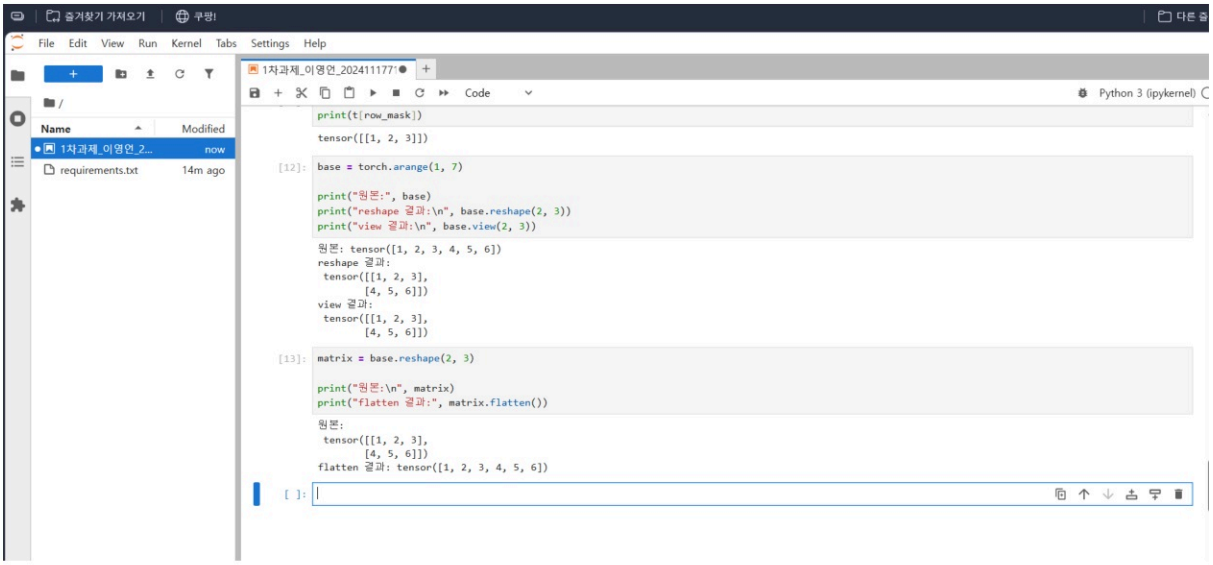
resize_(2,4)는 8개 값을 2행 4열 모양으로 바꾼다. unsqueeze(1)은 가운데 길이 1인 축을 넣어 (2,1,4)를 만들고, expand(2,2,4)는 그 축만 2로 확장한다. 두 번째 예도 (3,4)에서 (3,1,4), 다시 (3,5,4)가 된다. 확장 대상 차원이 기존에 1이어야 임의 크기로 늘릴 수 있다. resize_는 원본 Tensor의 크기저장공간을 직접 변경하는 in- place 연산이므로 reshape처럼 새 결과만 만든다고 오해하지 않도록 한다.

Part 4. Tensor 구조 변경

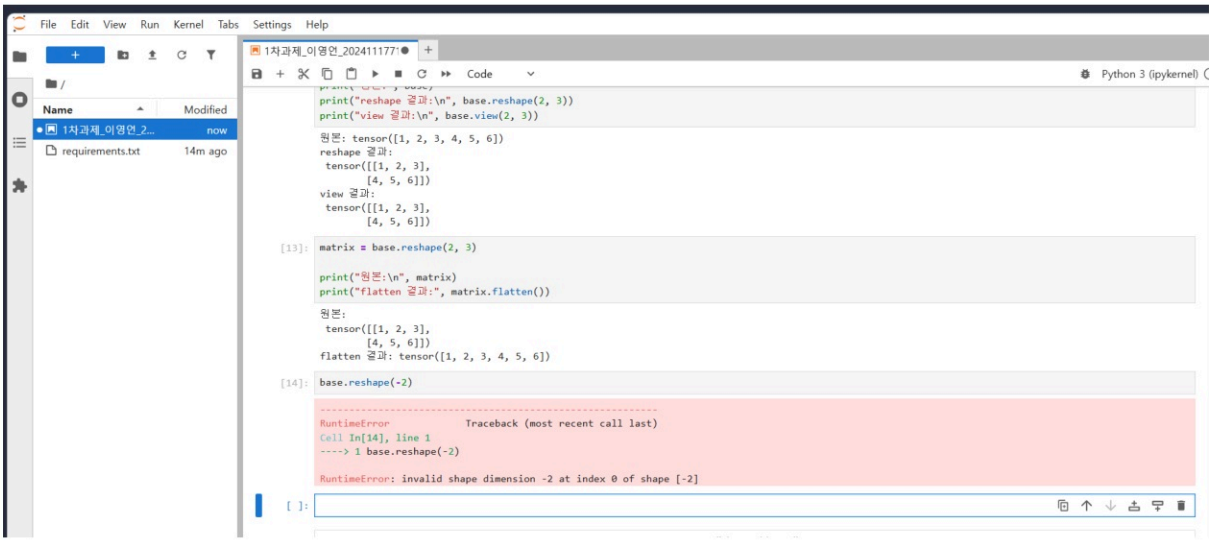
실행 화면 증거

reshape/view, flatten, unsqueeze/squeeze, 전치, permute, expand/repeat, resize_와 확장 과정의 화면이다.

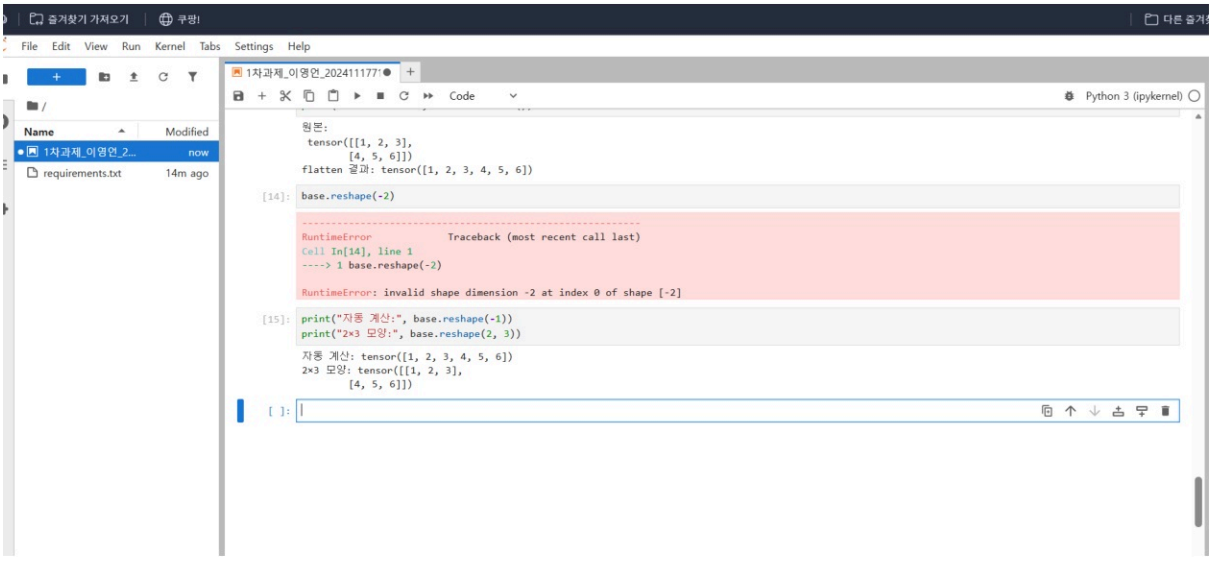
아래 캡처는 본인 Windows PowerShell 및 Jupyter Lab에서 실행한 화면이다. 코드와 출력, 오류 traceback을 보존했으며, 캡처 아래 설명은 해당 단계에서 확인한 결과와 원인을 요약한다.



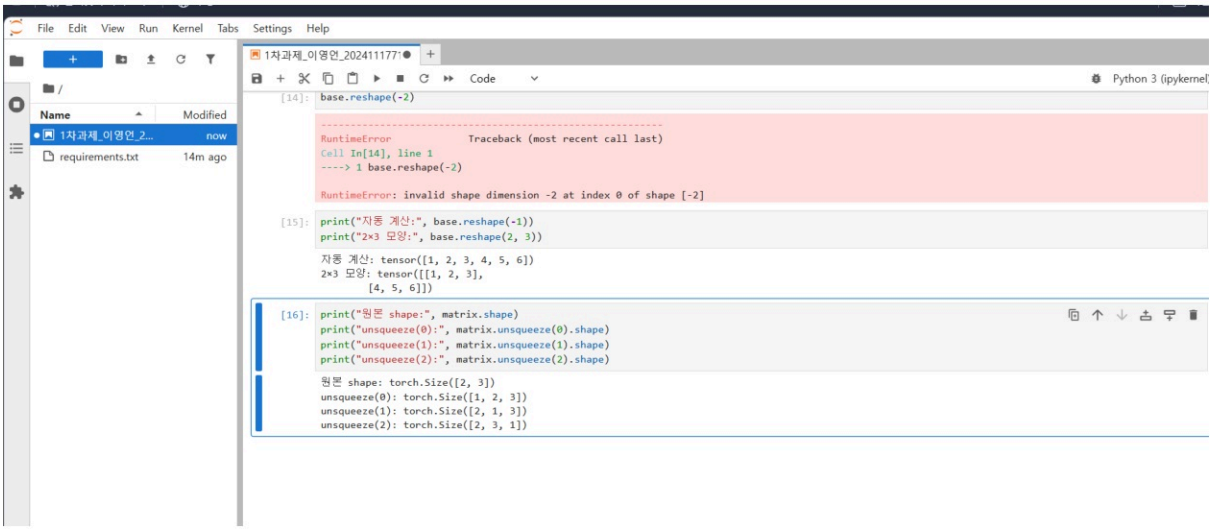
2×3 텐서를 `flatten()`으로 1차원으로 펼쳤다.



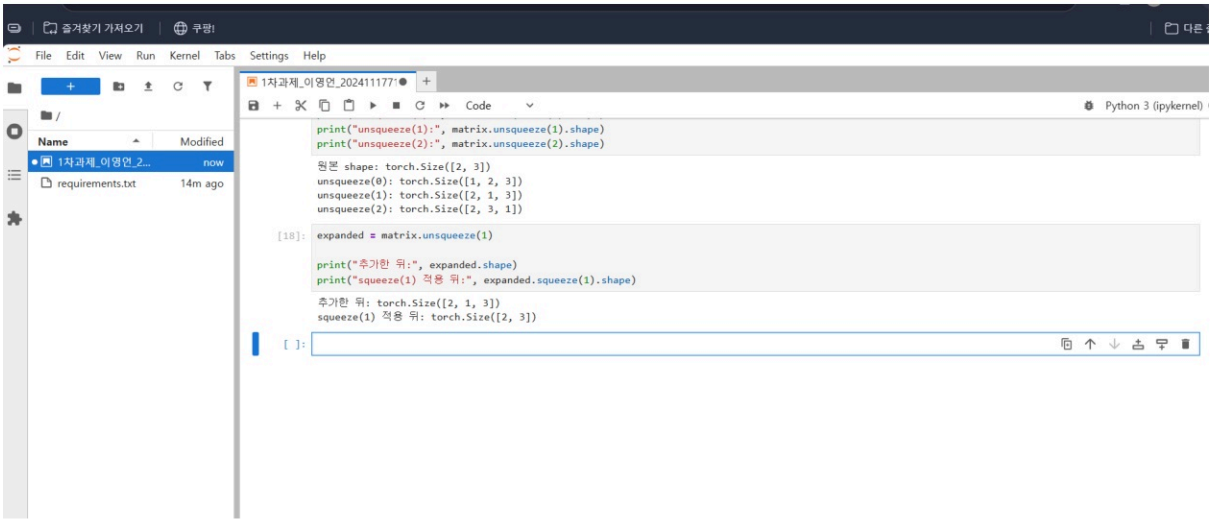
`reshape()`에 사용할 수 없는 크기 `-2`를 지정해 `RuntimeError`가 발생했다.



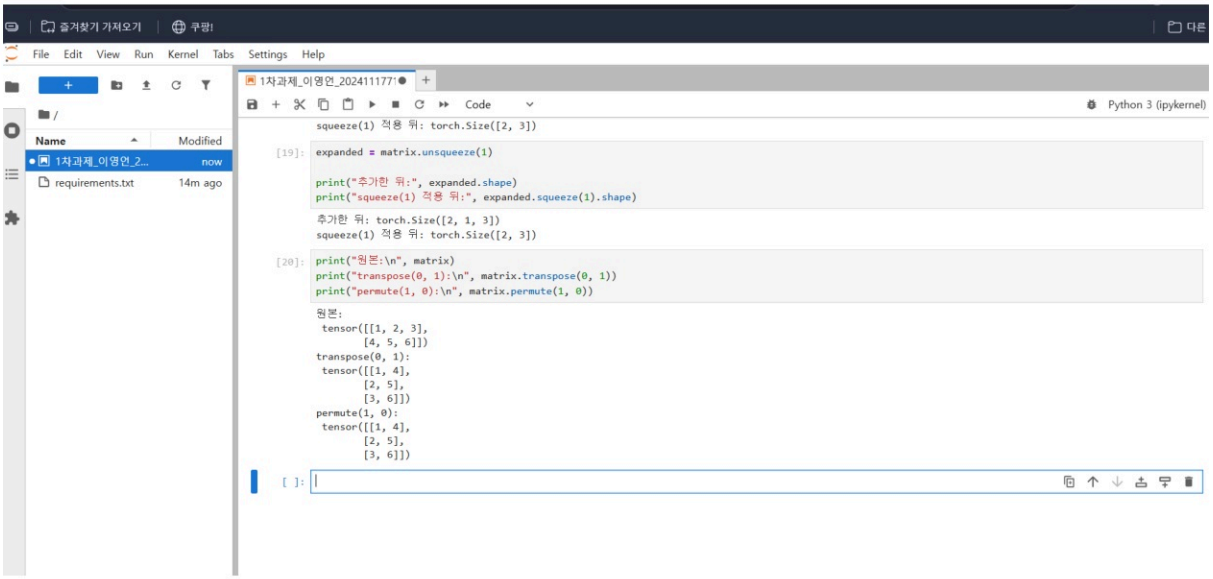
잘못된 `-2` 대신 `-1`을 사용해 텐서 크기를 자동 계산하고, 2×3 형태로 변경했다.



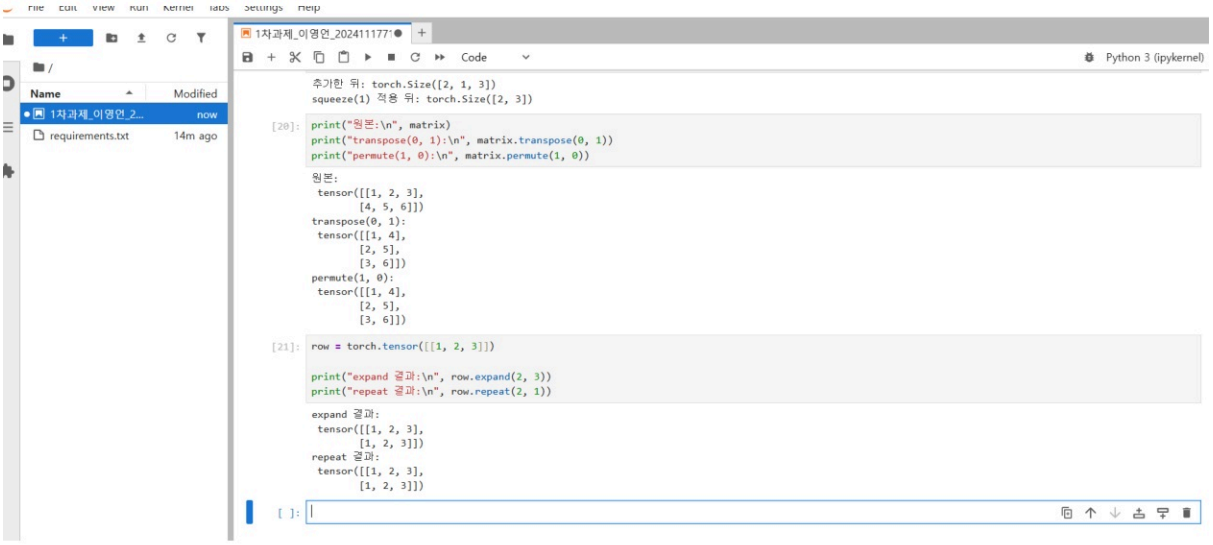
`reshape(-1)`로 크기를 자동 계산하고 `unsqueeze()`로 차원을 추가해 각 텐서의 shape을 확인했다.



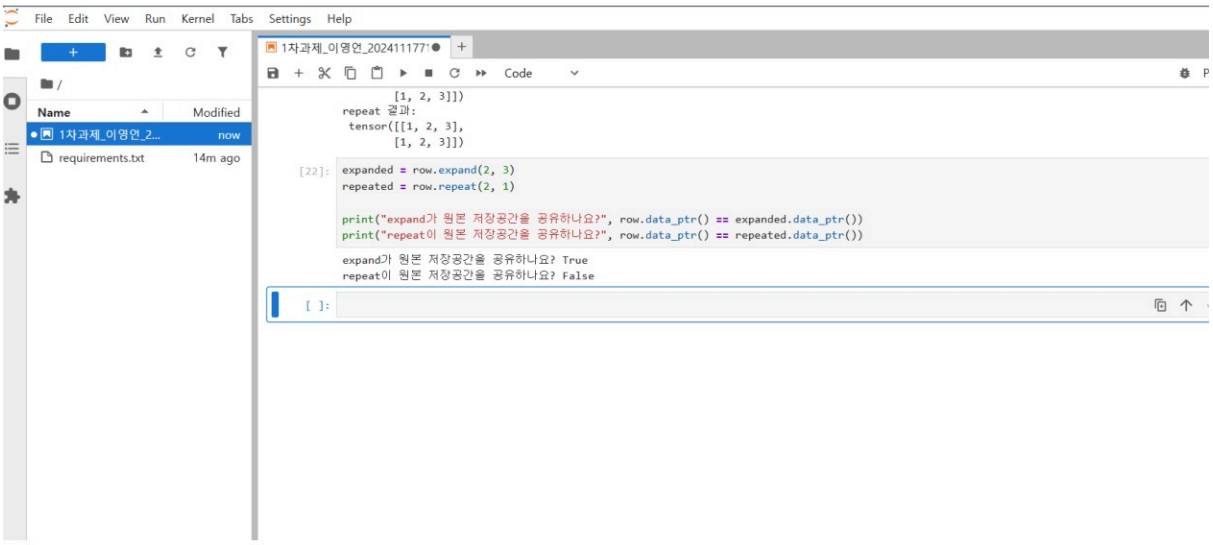
`unsqueeze()`의 위치에 따라 2×3 텐서에 차원을 추가하고 shape 변화를 확인했다.



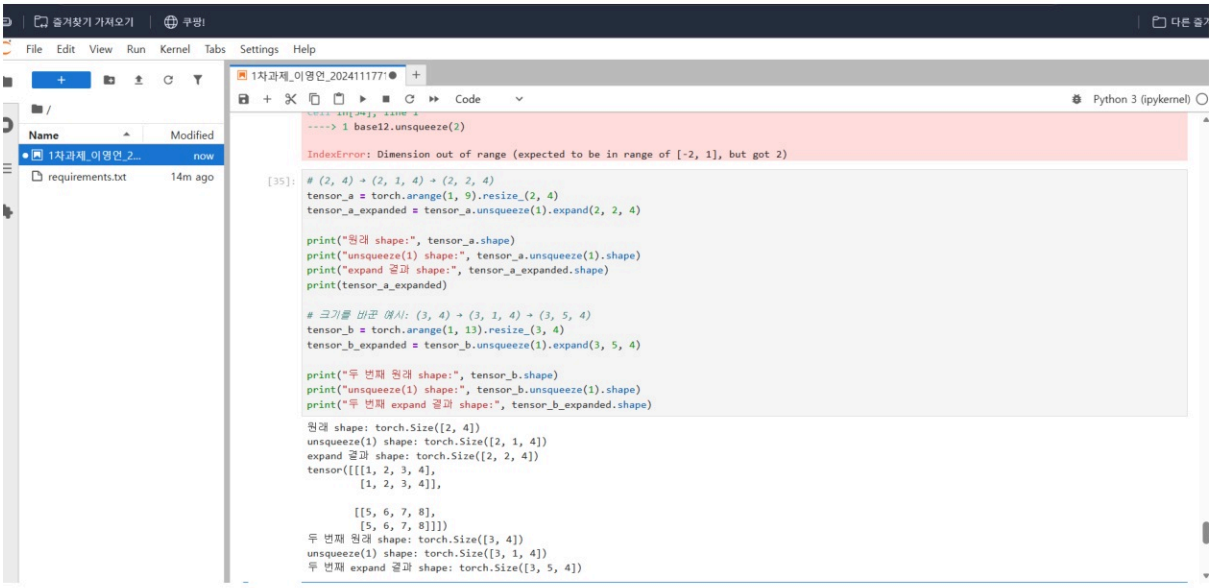
`transpose()`와 `permute()`로 행과 열의 위치를 바꾼 결과를 비교했다.



`expand()`와 `repeat()`으로 행을 늘려 결과를 비교했다



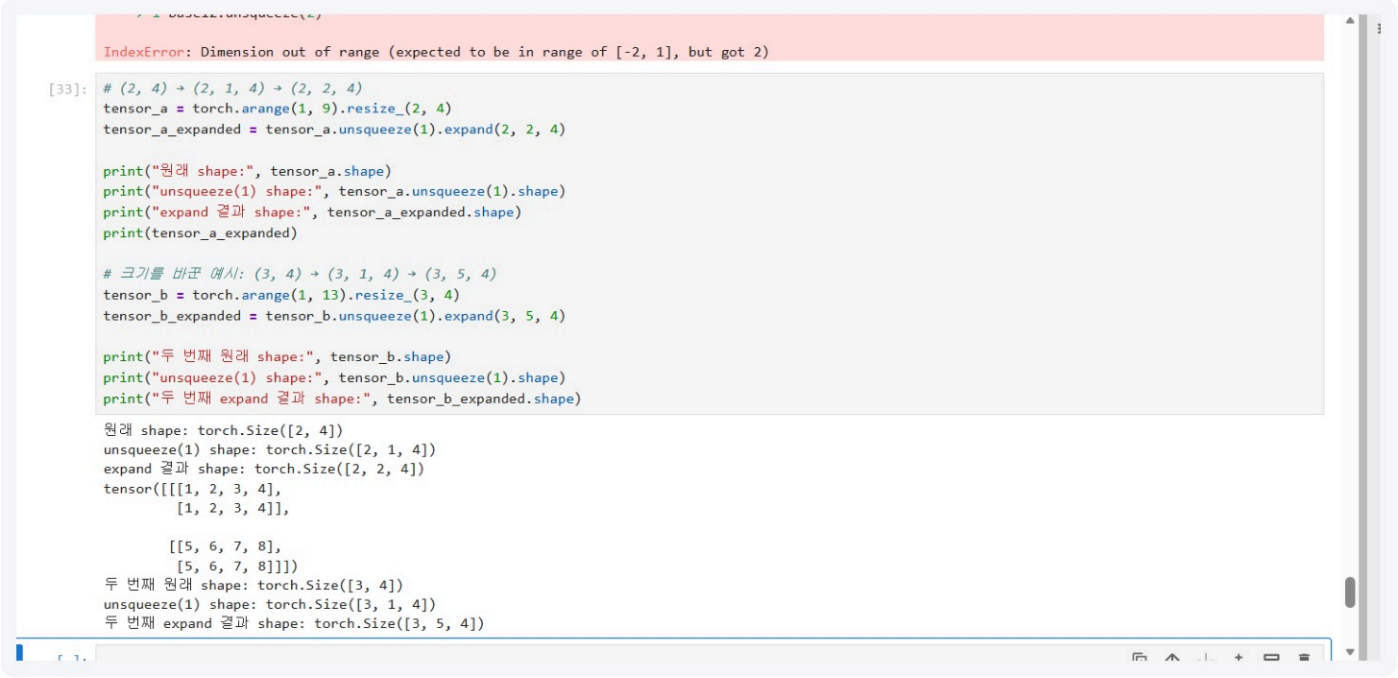
`data_ptr()`로 `expand()`와 `repeat()`의 원본 저장공간 공유 여부를 비교했다.



`resize_`로 텐서 모양을 바꾸고 `unsqueeze`로 차원을 추가한 뒤 `expand`로 반복 확장했다.

Part 4.6 실행 화면: resize_, unsqueeze, expand

JupyterLab에서 (2,4) → (2,1,4) → (2,2,4), (3,4) → (3,1,4) → (3,5,4) 확인



```
IndexError: Dimension out of range (expected to be in range of [-2, 1], but got 2)
```

[33]:

```
# (2, 4) → (2, 1, 4) → (2, 2, 4)
tensor_a = torch.arange(1, 9).resize_(2, 4)
tensor_a_expanded = tensor_a.unsqueeze(1).expand(2, 2, 4)

print("원래 shape:", tensor_a.shape)
print("unsqueeze(1) shape:", tensor_a.unsqueeze(1).shape)
print("expand 결과 shape:", tensor_a_expanded.shape)
print(tensor_a_expanded)

# 크기를 바꾼 예시: (3, 4) → (3, 1, 4) → (3, 5, 4)
tensor_b = torch.arange(1, 13).resize_(3, 4)
tensor_b_expanded = tensor_b.unsqueeze(1).expand(3, 5, 4)

print("두 번째 원래 shape:", tensor_b.shape)
print("unsqueeze(1) shape:", tensor_b.unsqueeze(1).shape)
print("두 번째 expand 결과 shape:", tensor_b_expanded.shape)
```

```
원래 shape: torch.Size([2, 4])
unsqueeze(1) shape: torch.Size([2, 1, 4])
expand 결과 shape: torch.Size([2, 2, 4])
tensor([[[1, 2, 3, 4],
         [1, 2, 3, 4]],

        [[5, 6, 7, 8],
         [5, 6, 7, 8]]])
두 번째 원래 shape: torch.Size([3, 4])
unsqueeze(1) shape: torch.Size([3, 1, 4])
두 번째 expand 결과 shape: torch.Size([3, 5, 4])
```

두 예제에서 크기 1인 축을 expand로 확장하고 결과 shape을 확인했다.

Part 5. 기본 연산

5.1 Dot product, 5.2 NumPy 벡터화와 broadcasting

5.1 세 방식의 내적과 PyTorch 내적

```
a = [1, 2, 3]
b = [4, 5, 6]
dot_zip = sum(x*y for x,y in zip(a,b))
dot_index = sum(a[i]*b[i] for i in range(len(a)))
a_np, b_np = np.array(a), np.array(b)
np.dot(a_np, b_np)  # 32
a_np @ b_np        # 32
a_t, b_t = torch.tensor(a), torch.tensor(b)
torch.dot(a_t, b_t)  # tensor(32)
a_t @ b_t            # tensor(32)
```

내적은 같은 위치 원소끼리 곱한 뒤 모두 더한다. 계산은 1× 4 + 2× 5 + 3× 6 = 32다. zip 반복문, 인덱스 반복문, NumPy의 dot과 @, PyTorch의 dot과 @가 모두 32를 출력했다. @는 두 벡터의 내적을 표현하는 행렬 곱 연산자다.

5.2 원소별 벡터 연산

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
a + b       # [5, 7, 9]
a * b       # [4, 10, 18]
a ** 2      # [1, 4, 9]
np.exp(a)   # [2.718..., 7.389..., 20.085...]
np.log(a+1) # [0.693..., 1.098..., 1.386...]
```

NumPy 배열의 +, *, **는 각 위치끼리 한 번에 계산된다. exp는 자연상수 e의 거듭제곱, log는 자연로그이며 여기서 log(a+1)은 0으로 로그를 계산하는 것을 피하도록 1을 더했다. for문으로 계산한 원소별 덧셈 [5,7,9]와 배열 연산 결과가 같음을 비교했다.

2차원 열과 1차원 행의 broadcasting

```
column = np.array([[1], [2], [3]])  # (3,1)
row = np.array([10, 20, 30])          # (3,)
column + row
# [[11, 21, 31], [12, 22, 32], [13, 23, 33]]
```

끝에서부터 차원 크기를 비교할 때 한쪽 크기가 1이면 그 축으로 복제된 것처럼 계산한다. (3,1)과 (3,)의 덧셈은 (3,1)+(1,3)처럼 맞춰져 결과 shape (3,3)이 된다. 각 행에는 열 벡터의 한 값이 더해지고 각 열에는 row의 값이 더해진다.

Part 5. 기본 연산

5.3 100만 원소 성능 비교, 5.4 PyTorch 연산과 GPU 대체

5.3 %timeit 성능 비교와 이유

```
data_list = list(range(1_000_000))
data_array = np.arange(1_000_000)
%timeit [x ** 2 for x in data_list]
%timeit data_array ** 2
```

캡처에서는 리스트 컴프리헨션이 약 107 ms, NumPy가 약 3.99 ms를 기록했다. 이 실행 환경에서는 NumPy가 약 27배 빠른 측정이었다. 수치는 컴퓨터와 부하, 반복 조건에 따라 달라지지만 NumPy가 보통 더 빠른 이유는 원소마다 Python 인터프리터가 반복하는 작업을 줄이고, 연속된 배열을 C 기반 벡터화 루프로 처리하기 때문이다. 평가에서는 특정 시간 숫자보다 이 경향과 이유가 중요하다.

5.4 PyTorch 원소별 연산과 broadcasting

```
a_t = torch.tensor([1., 2., 3.])
b_t = torch.tensor([4., 5., 6.])
a_t + b_t
a_t * b_t
a_t ** 2
column_t = torch.tensor([[1.], [2.], [3.]])
row_t = torch.tensor([10., 20., 30.])
(column_t + row_t).shape  # torch.Size([3, 3])
```

PyTorch Tensor도 같은 모양의 Tensor끼리 원소별 사칙연산과 거듭제곱을 지원한다. (3,1)과 (3,)의 더하기는 NumPy와 같은 broadcasting 규칙을 적용해 (3,3)이 된다. CPU에서 수행해도 수식과 결과 shape은 같다.

GPU가 없는 환경에서 x @ w 수행

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
x = torch.randn(10000, 128, device=device)
w = torch.randn(128, device=device)
result = x @ w
# CUDA 사용 불가 캡처: device=cpu, result shape=(10000,)
```

x의 각 행은 길이 128의 벡터이고 w도 길이 128이므로 행마다 내적 하나가 계산된다. 입력 shape (10000,128)과 (128,)의 결과는 (10000,)이다. 캡처에서 CUDA 사용 가능 여부가 False여서 과제의 분기 코드가 CPU를 선택했다. CUDA 장치가 없어 이 항목을 GPU로 계산하지 않고 CPU에서 동일한 식과 결과 shape을 확인한 것은 과제의 명시적 대체 지침에 따른다.

> GPU 미사용은 결과 값이 GPU에서 계산되지 않았다는 점만 다르다. 장치 선택, 행렬 곱 규칙, 출력 shape 설명은 동일하게 적용된다.

Part 5. 기본 연산

5.5 행렬 곱 규칙과 순서, 5.6 Softmax 및 overflow 안정화

5.5 행렬 곱셈의 안쪽 차원 규칙

```
A = torch.randn(100, 128)
B = torch.randn(128, 256)
(A @ B).shape  # (100, 256)
wrong_W = torch.randn(256, 128)
A @ wrong_W    # RuntimeError: 128 != 256
fixed = A @ wrong_W.T  # (100, 256)
```

행렬 (m,n)과 (n,p)를 곱하면 결과는 (m,p)다. 첫 행렬의 열 수와 두 번째 행렬의 행 수, 즉 안쪽 차원이 같아야 한다. (100,128)@(128,256)은 안쪽 128이 일치해 (100,256)이 된다. (100,128)@(256,128)은 128과 256이 달라 RuntimeError다. wrong_W.T의 shape은 (128,256)이므로 안쪽 차원을 맞춰 오류를 해결한다.

곱하는 순서에 따라 shape이 달라짐

```
X = torch.randn(50, 100)
W = torch.randn(100, 50)
X @ W  # (50, 50)
W @ X  # (100, 100)
```

행렬 곱은 일반적으로 교환법칙이 성립하지 않는다. 여기서는 두 순서의 결과 shape부터 (50,50)과 (100,100)으로 다르므로 같은 결과일 수 없다. 행렬 곱의 순서는 입력의 의미와 차원 규칙에 따라 유지해야 한다.

5.6 Softmax 기본 계산

```
logits = np.array([2.3, 5.1, 0.7])
softmax = np.exp(logits) / np.exp(logits).sum()
# [0.05666832, 0.93189054, 0.01144414]
# 각 확률의 합은 약 1.0
```

Softmax는 각 입력을 지수화한 뒤 전체 지수합으로 나누어 확률처럼 합이 1인 값으로 바꾼다. 값이 큰 5.1에 해당하는 두 번째 결과가 약 0.932로 가장 크다. 출력의 합이 1에 가까운 것은 정규화가 올바르게 이루어졌음을 보여준다.

큰 logits에서 overflow가 발생하는 이유와 안정화

```
large = np.array([1000, 2000, 3000])
unstable = np.exp(large) / np.exp(large).sum()
# overflow warning, [nan, nan, nan]
shifted = large - np.max(large)  # [-2000, -1000, 0]
stable = np.exp(shifted) / np.exp(shifted).sum()
# [0., 0., 1.], sum=1.0
```

exp(1000)처럼 너무 큰 값을 float64 범위로 표현할 수 없어 inf가 생기고, inf/inf가 nan이 된다. 모든 logits에서 최댓값을 빼도 Softmax 비율은 변하지 않는다. exp(x-c)/sum(exp(x-c))에서 공통인 exp(-c)가 분자와 분모에서 약분되기 때문이다. 최댓값 3000을 빼면 [2000,1000,0]이 되어 지수 계산이 안정되고, 결과 비율은 원래 Softmax와 같으며 합이 1이다.

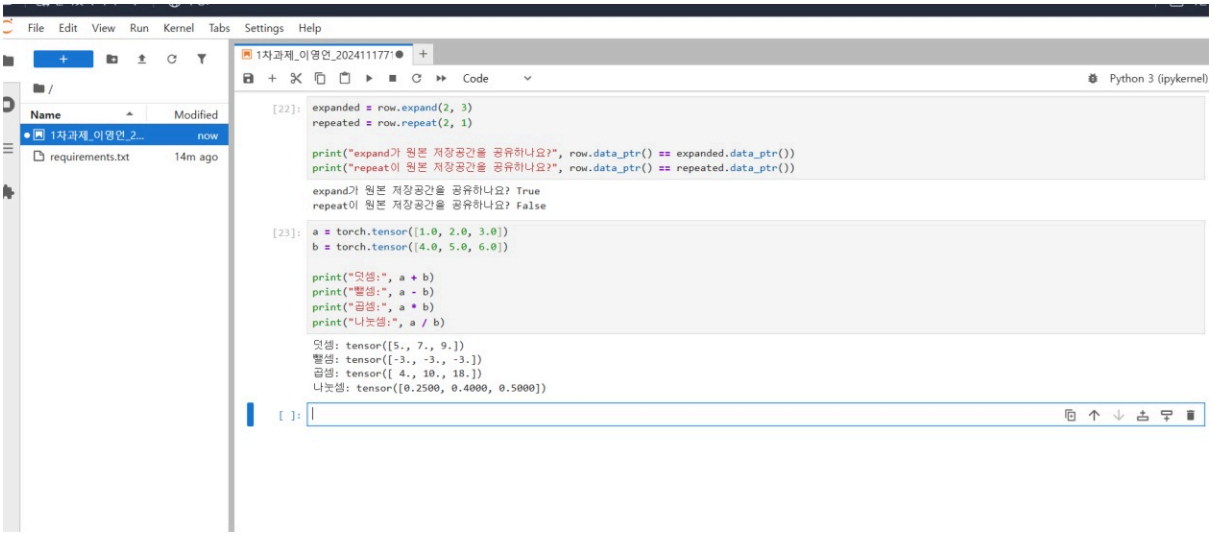
> 고정된 logits에서는 가장 큰 값의 확률이 반올림 결과 1.0, 나머지는 0.0으로 표시될 수 있다. 이는 안정화 이후의 정상적인 극단 비율 결과다.

Part 5. 기본 연산

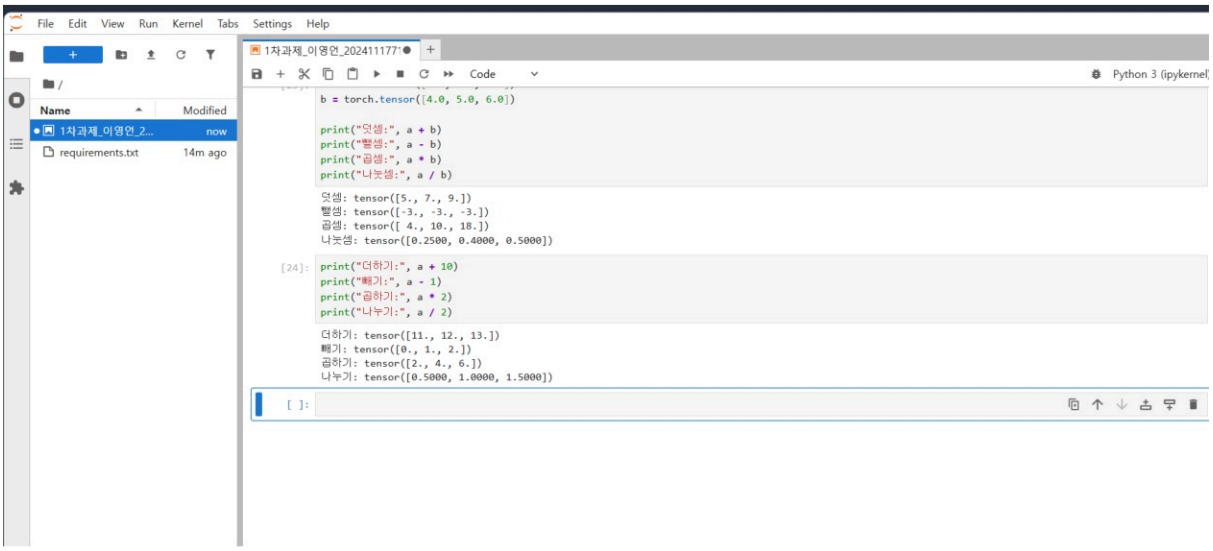
실행 화면 증거

내적, 벡터화, broadcasting, %timeit, PyTorch 행렬 곱, 행렬 곱 규칙 오류, Softmax overflow와 안정화 결과 화면이다.

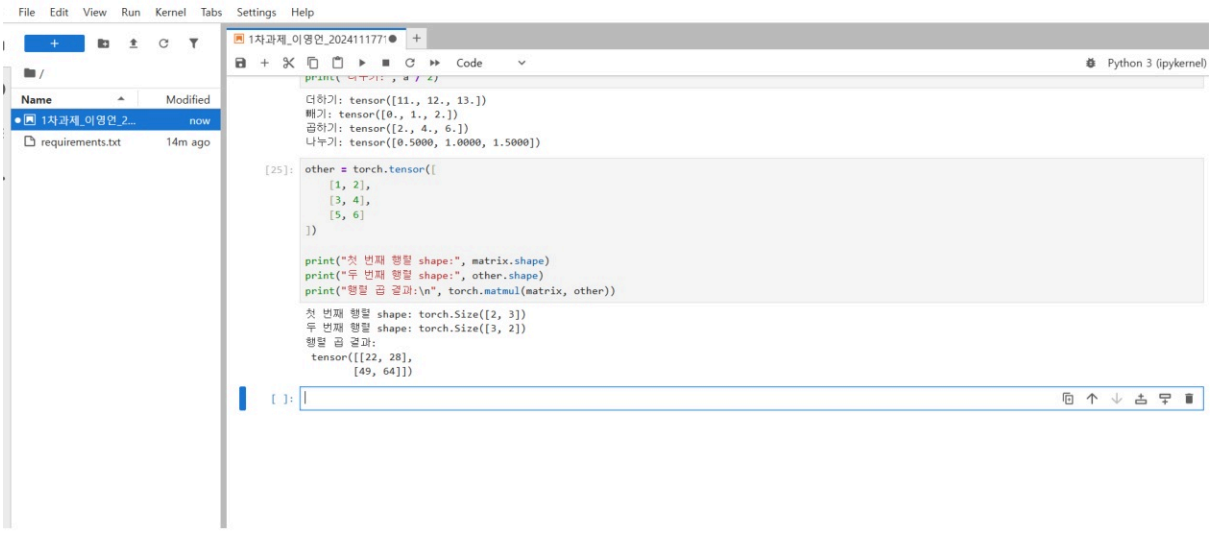
아래 캡처는 본인 Windows PowerShell 및 Jupyter Lab에서 실행한 화면이다. 코드와 출력, 오류 traceback을 보존했으며, 캡처 아래 설명은 해당 단계에서 확인한 결과와 원인을 요약한다.



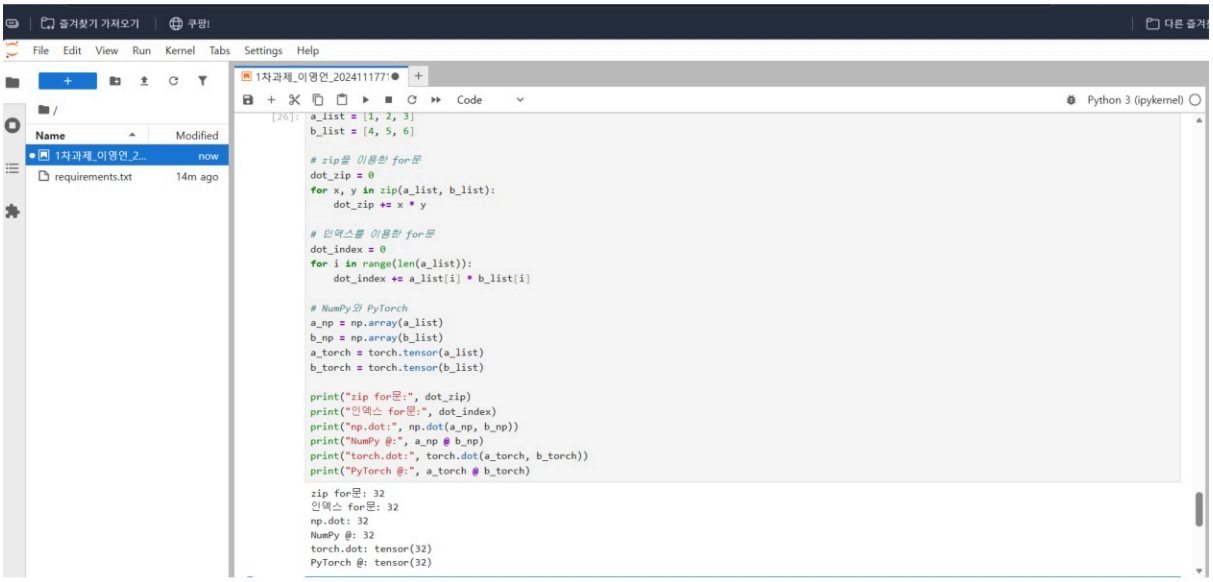
두 텐서의 같은 위치에 있는 원소끼리 사칙연산을 수행했다."**



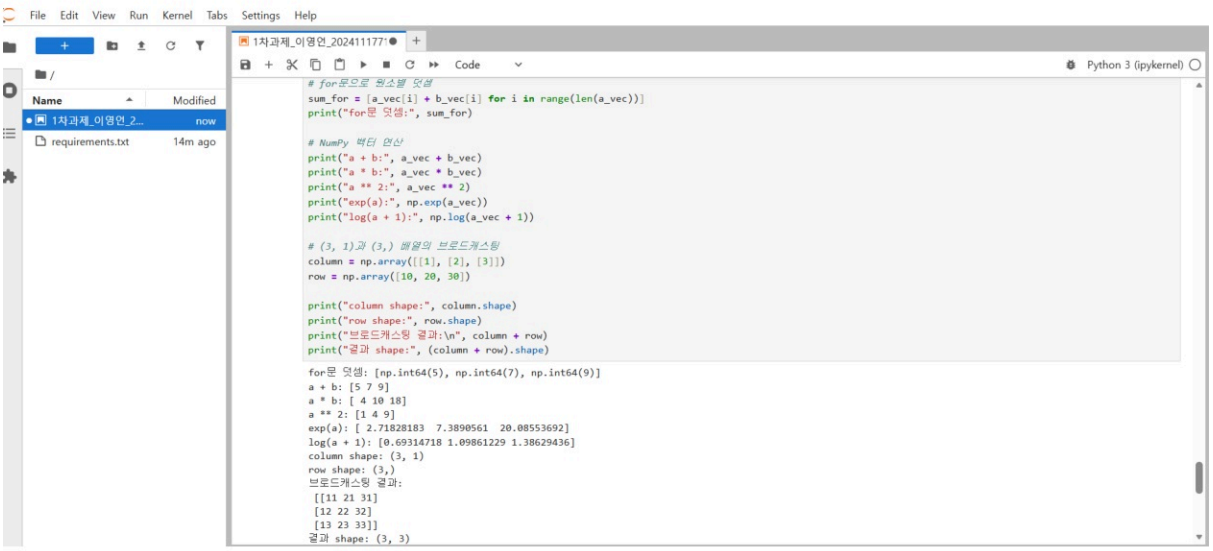
스칼라 값을 텐서의 모든 원소에 적용해 사칙연산 결과를 확인했다.



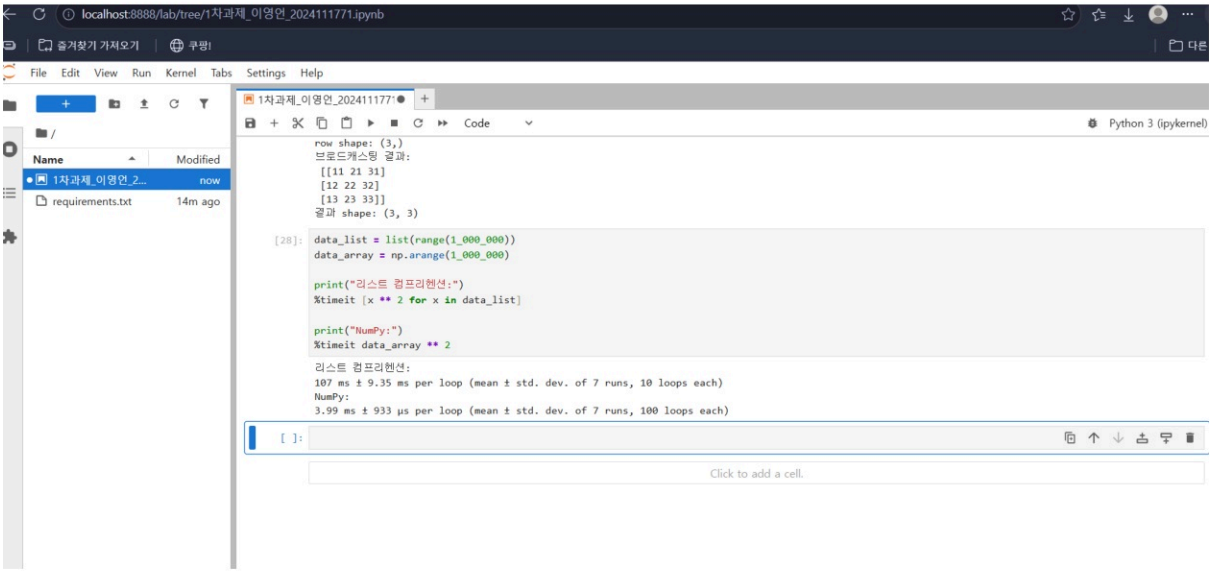
안쪽 차원이 일치하는 두 행렬을 곱해 shape이 `(2, 2)`인 결과를 확인했다.



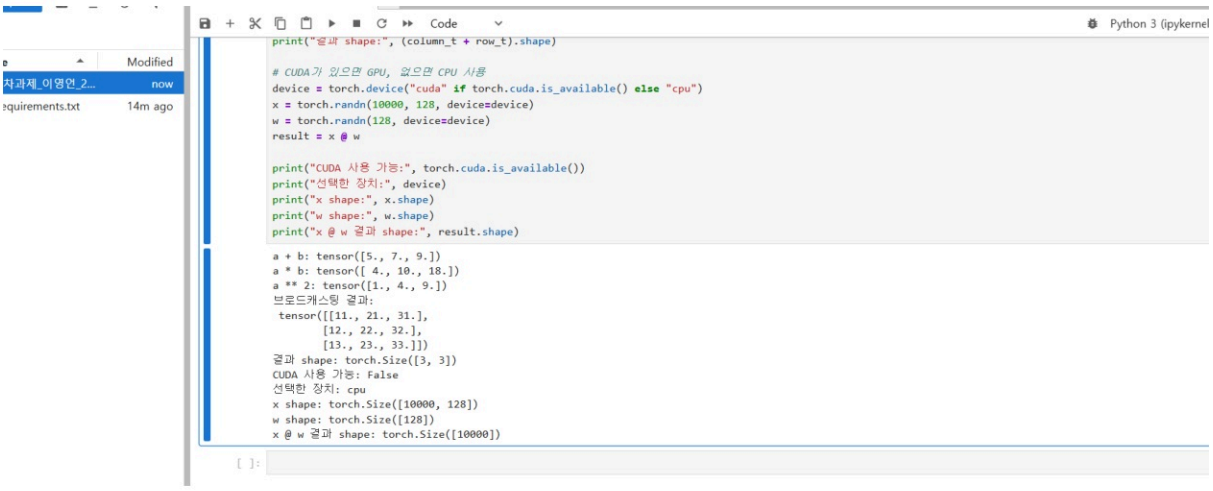
반복문·NumPy·PyTorch 방식으로 내적을 계산하고 결과가 모두 32임을 확인했다.



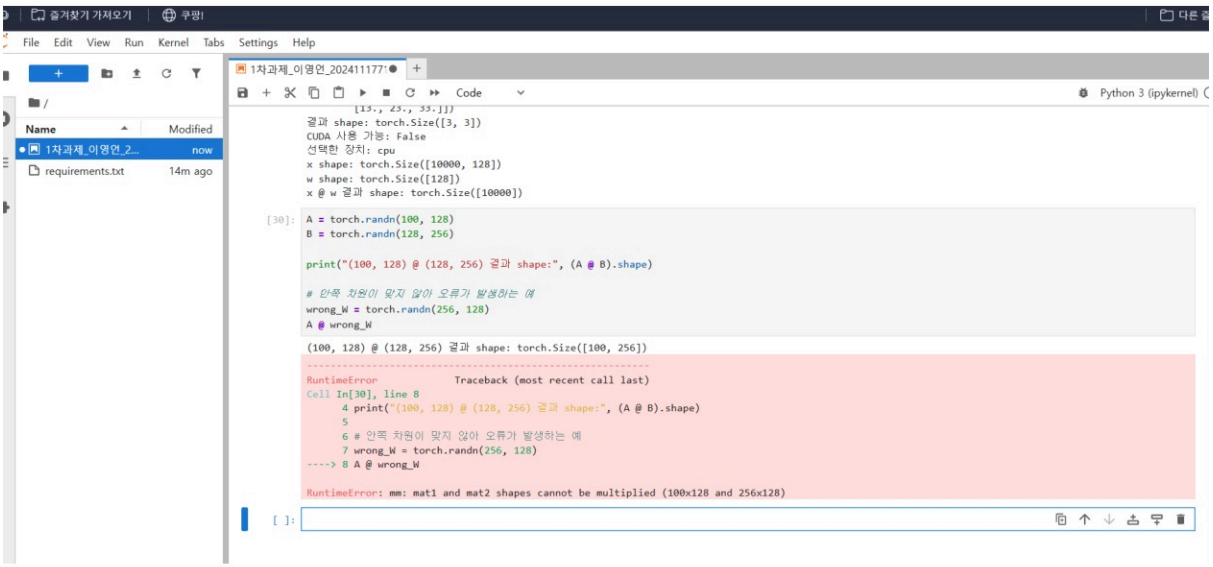
NumPy 벡터 연산과 브로드캐스팅을 확인했다. `(3, 1)`과 `(3,)` 배열의 덧셈 결과는 `(3, 3)`이다.



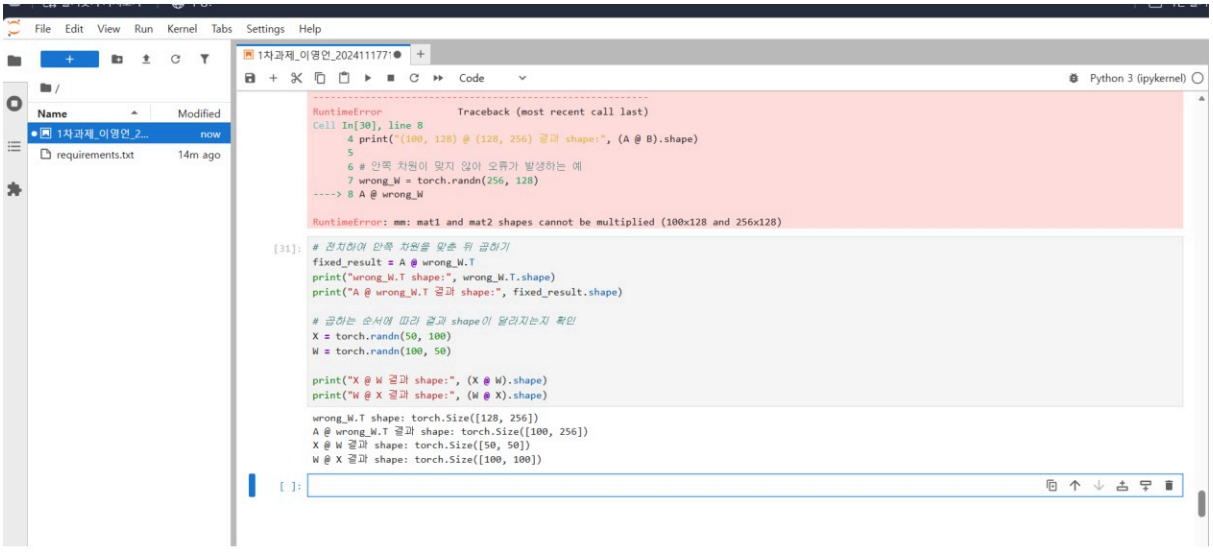
100만 개 데이터의 제곱 연산을 비교한 결과, NumPy의 벡터화 연산이 리스트 컴프리헨션보다 빠르게 실행됐다.



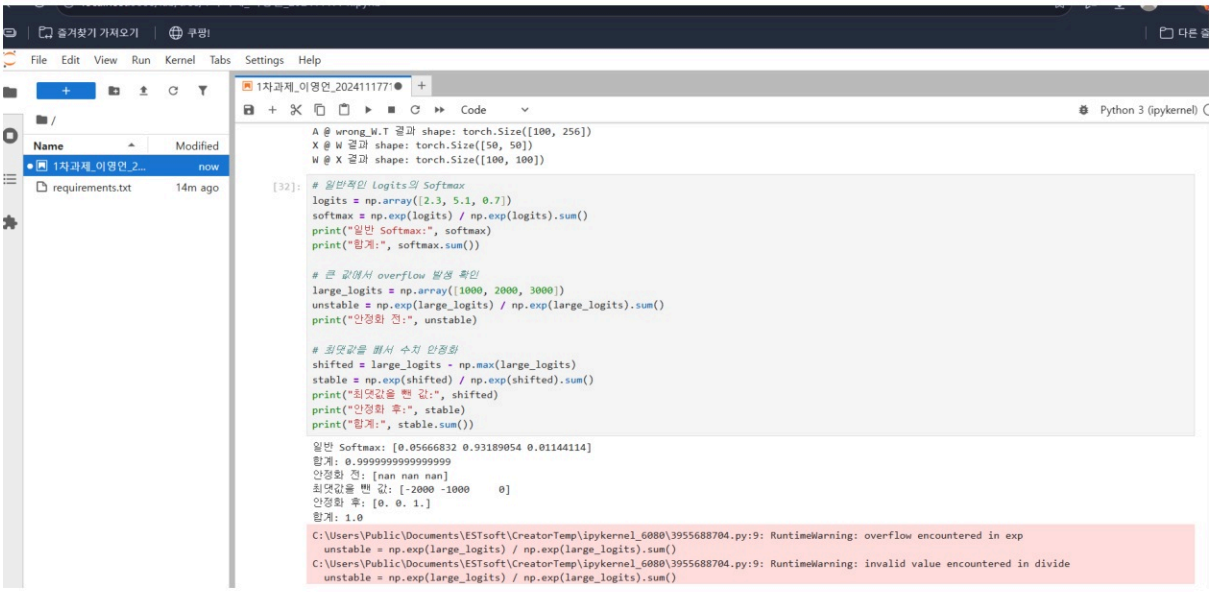
PyTorch에서도 원소별 연산과 브로드캐스팅이 가능함을 확인했다. CUDA를 사용할 수 없어 CPU에서 계산했으며, 큰 텐서 곱의 결과 shape은 `(10000,)`이다.



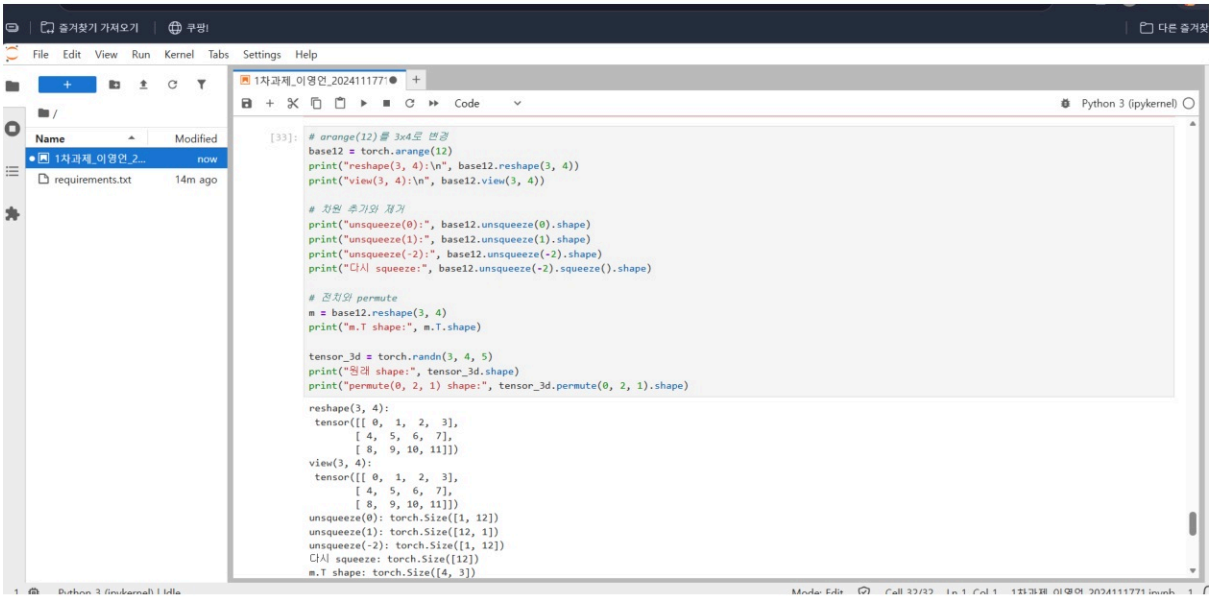
행렬 곱에서 안쪽 차원이 다르면 RuntimeError가 발생한다. `(100, 128) @ (128, 256)`은 가능하지만 `(100, 128) @ (256, 128)`은 불가능하다.



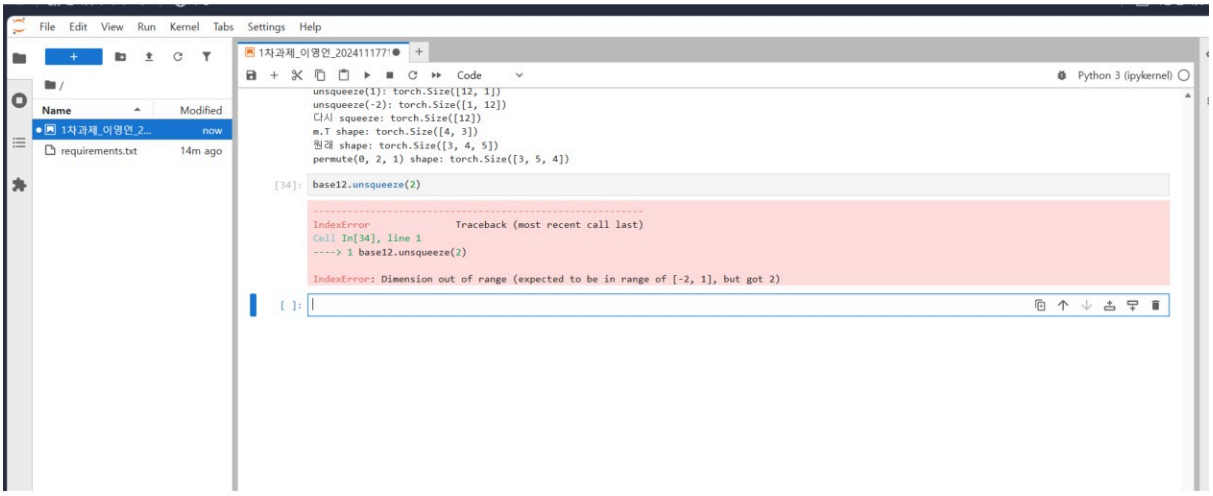
전치로 행렬 곱셈 오류를 해결했다. 또한 `X @ W`와 `W @ X`의 결과 shape이 달라 행렬 곱은 교환법칙이 성립하지 않음을 확인했다



큰 입력값을 그대로 지수화하면 overflow로 `nan`이 발생한다. logits에서 최댓값을 빼는 방식으로 안정화해 정상적인 Softmax 결과를 확인했다.



큰 입력값을 그대로 지수화하면 overflow로 `nan`이 발생한다. logits에서 최댓값을 빼는 방식으로 안정화해 정상적인 Softmax 결과를 확인했다.



1차원 텐서에 허용 범위를 벗어난 `unsqueeze(2)`를 적용해 IndexError를 재현했다.

마지막 정리. 실행 환경과 제출 확인

환경은 캡처에서 확인된 사실만 기록

항목	확인 결과
OS	Windows (세부 버전은 캡처에 표시되지 않음)
GPU / 모델	CUDA 사용 가능 장치 없음으로 확인; 물리 GPU 모델명은 캡처에 없어 미확인
Python	3.11.9, .venv 가상환경
PyTorch	2.15.0+cpu, 캡처 출력 기준
CUDA	사용 불가; torch.version.cuda 출력 None (CPU 빌드)
실습 device	cpu

CUDA 항목 대체 수행에 대한 설명

torch.cuda.is_available()이 False이고 torch.version.cuda가 None이며 nvidia-smi도 실행되지 않았다. 과제 유의사항에 따라 GPU 장치명 조회와 .to("cuda"), CPU-CUDA 혼합 연산, CUDA 계산을 강제로 실행하지 않았다. 대신 CUDA 사용 불가 출력과 device = "cuda" if torch.cuda.is_available() else "cpu" 조건 코드를 제시해 CPU에서 x @ w를 계산했다. 이 환경에서 직접 만들 수 없는 CPU-CUDA 장치 불일치 오류는 재현하지 못했으며, 원인과 동일 장치로 옮기는 해결 원리를 본문에서 설명했다.

제출 파일 및 최종 점검

- 보고서 파일명: 1차과제_실습보고서_이영언_2024111771.pdf
- 노트북 파일명: 1차과제_이영언_2024111771.ipynb
- 노트북 제출 조건: Part 2~5 코드와 출력 포함, Restart Kernel and Run All 완료, 셀 번호 순서 확인
- 보고서 제출 조건: Part 1~5 순서, 명령코드결과 캡처해설, 오류 원인 및 해결 결과, requirements.txt 전문, 실행 환경 표

평가 비중은 환경 구축 20%, 실습 완성도 40%, 설명의 정확성 30%, 보고서 형식 10%다. 이 보고서는 실제 캡처를 기준으로 작성했으며, CUDA 미사용은 과제의 GPU 없는 환경 대체 지침을 따랐다.